UniRL: Trajectory-Centric Post-Training Across Heterogeneous Generative Models

UniRL Authors *authors@to.be.added*
Affiliations to be added before the arXiv release

Abstract

Reinforcement-learning systems share an outer loop—generate, score, assign credit, and update—yet their implementations fragment because autoregressive (AR) tokens, diffusion trajectories, multimodal values, and multi-stage rollouts carry different state and impose different execution constraints. We present UniRL, a trajectory-centric system that places a lineage-preserving, modality-aware intermediate representation (IR) at the boundary between post-training semantics and distributed execution. Each trajectory is a prompt-rooted tree of typed generation layers, recording decoded outputs, conditioning inputs, stage-specific replay segments, rewards, and the behavior version of each layer. Three components share that representation: rollout engines fill each generation layer, scoring propagates rewards and credit along tree ancestry, and training backends replay each stage's objective under a configured placement, tensor transport, weight publication, and memory residency. These interfaces keep model-specific replay separate from lineage tracking, data movement, and scheduling, without collapsing heterogeneous objectives or engine–topology pairs into one kernel. We implement this design in UniRL v0.2.0 and examine its reward-serving and rollout-engine boundaries through two image-editing case studies on generation–reward overlap and conditioning fidelity. Diffusion post-training improves held-out automatic scores, while an eight-GPU native-recipe comparison reports lower iteration time than VeRL-Omni. In a recipe-matched 500-rollout Qwen3-4B-Base Group Relative Policy Optimization (GRPO) comparison, UniRL records 8.2% higher end-to-end output-token throughput than veRL, and a source-importing CPU harness passes ten core IR checks.

1 Introduction

Online post-training is a feedback program. It expands inputs into policy samples, evaluates those samples, turns feedback into a learning signal, and applies one or more optimizer updates. At this level, language-model Group Relative Policy Optimization (GRPO), diffusion-model policy gradients, prompt-rewrite pipelines, and tool-using agents look deceptively similar. Their data paths do not. An autoregressive (AR) update may need packed token IDs, masks, rollout log probabilities, and per-token values; a diffusion update may need a sparse set of latent states, a noise schedule, and transition statistics; a composed workflow must retain which image descended from which rewritten prompt; an asynchronous run must additionally retain which behavior version generated every sample.

Existing systems have made major progress on scalable LLM post-training and flexible device mapping (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025). Fully or partially asynchronous systems further address rollout tail latency and accelerator idle time (Fu et al., 2025; Zhang et al., 2026; Zhu et al., 2025). Post-training is now expanding from single-stage text generation to iterative diffusion denoising and multi-stage multimodal pipelines (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025; Huang et al., 2026). Those stages differ in modality, replay state, and branching factor while still sharing ancestry for conditioning and credit assignment, so flat LLM-specific tensor dictionaries no longer suffice. The remaining design question is how the information shared by these workflows should survive changes in their model-specific state and execution paths.

Can one post-training program cross generative model families and execution topologies while retaining the structure needed for correct grouping, replay, credit assignment, transport, and behavior-version control?

Our answer in UniRL is to introduce a *trajectory intermediate representation (IR)*—a representation for stages that differ in modality and batch size—and expose execution choices as explicit contracts around that representation. Rather than treating an individual row or an LLM-specific tensor dictionary as the unit of scheduling and sharding, the IR organizes each batch as a forest of prompt-rooted trees whose width grows when a stage branches. Within each layer, typed leaves separate decoded multimodal values, model conditions, and per-stage replay segments. Distributed methods declare how each structured field scatters, concatenates, packs, or remains shared, and dense tensors can be replaced by row-addressable references without changing the trajectory structure. This representation flows through a small set of logical roles that can be mapped onto colocated or separate *GPU slabs* (contiguous subsets of accelerators allocated to a role).

The paper develops three coupled design ideas:

- A *lineage-preserving trajectory IR* in which a rollout request is a typed tree skeleton spanning AR and diffusion stages, each generated layer is conditioned through its ancestry and keeps its own replay segment, and training folds reward and credit back along the same tree (Sections 3.1–3.5).
- A *modular interface for stage replay and optimization* that separates trajectory layers from stage-specific losses and distributed training stacks, so N_{mod} model bundles and N_{alg} shared algorithms require $N_{\text{mod}} + N_{\text{alg}}$ integrations rather than $N_{\text{mod}} \times N_{\text{alg}}$ (Sections 3.6 and 5).
- A *factorization of semantic roles from physical execution*: rollout remains one `Sample` $\rightarrow$ `Sample` fill of the trajectory skeleton, while field-aware dispatch, tensor references, placement, weight publication, and memory residency govern how those roles run on colocated, separate, and asynchronous topologies (Section 4), with two case studies examining reward-serving isolation/overlap and rollout-engine conditioning fidelity (Sections 6 and 7).

Asynchronous execution is where the third idea pays off most directly. Because rollout, reward, and training are logical roles rather than fixed steps of one loop, the orchestration layer—admission, root-complete grouping, version stamping, lag filtering, and weight publication—can be lifted out of individual trainers. UniRL's AR and diffusion trainers therefore share one bounded-staleness manager over the same trajectory IR (Section 4.5), and we exercise it on both workloads. For AR, single-seed capacity measurements with overlap enabled show lower per-rollout time than synchronous colocated rollout; the accompanying quality pilot did not pass, so no time-to-quality benefit is claimed (Section 8.4). For diffusion, denoising and replay compute dominate each iteration and leave little generation idle time for overlap to recover, so asynchrony yields no significant gain.

The evidence is organized from structural correctness, through credible learning and a composed workflow, to measured system cost. The experiment IDs and corresponding artifacts are specified in Section 8.

The current evidence comprises an archived CPU regression, reported GPU results for diffusion training, native-recipe systems performance and deployment topology, and a newly regenerated recipe-matched AR systems comparison. The diffusion result includes a material diversity regression; the diffusion systems comparison retains estimator and trainable-parameter differences, while the AR comparison contains only one process-level run per framework. These observations do not establish a cross-workload or time-to-quality advantage or complete the trajectory-centric claim: real-rollout semantic validation and a paired measurement of representation cost remain necessary.

2 Problem: Preserve Semantics While Changing Execution

Let an online post-training iteration consume a batch of initial root inputs X_0, such as text prompts or multimodal contexts, sample trajectories with behavior policy π_v, evaluate rewards R via reward operator

$\mathcal{R}$, construct credit A via credit-assignment operator $\mathcal{G}$, and update parameters θ:

$$X_0 \xrightarrow{\pi_v} \mathcal{T} \xrightarrow{\mathcal{R}} (\mathcal{T}, R) \xrightarrow{\mathcal{G}} (\mathcal{T}, A) \xrightarrow{U_\theta} \theta'. \tag{1}$$

The compact notation hides three forms of heterogeneity.

Trajectory heterogeneity. The sufficient statistics for an update depend on the generative process. For AR policies, a replay segment may be a variable-length packed token sequence with masks, old log probabilities, values, and returns. For stochastic differential equation (SDE) diffusion or flow policies, a replay segment may contain selected latent snapshots, a shared noise schedule, the indices of sampled transitions, and rollout or replay transition statistics. Multi-stage and agentic paths add nonuniform lineage and observations.

For example, let prompt p produce rewrites $w_{p,j}$ and images $x_{p,j,k}$. The image model conditions on $w_{p,j}$, but credit may compare either images under that rewrite or all images under p. A flattened image batch can implement both choices only if it retains or reconstructs the relevant parent keys. This example motivates explicit ancestry, while the image model's replay state motivates typed replay segments; neither requirement follows from the other.

Role heterogeneity. Generation, reward evaluation, replay, gradient computation, and weight publication differ in runtime, parallel layout, and lifecycle, not only in kernels. Rollout may run inside the training process or in a dedicated serving engine with its own tensor-parallel (TP) layout, such as SGLang (Zheng et al., 2024), vLLM (Kwon et al., 2023), or vLLM-Omni (Yin et al., 2026), while a sharded training backend such as FSDP (Zhao et al., 2023) or VeOmni (Ma et al., 2025a) owns parameters and optimizer state. Reward ranges from an in-process scorer to an environment-isolated scorer process or a remote multi-model service. These roles must agree on when each copy of the policy is resident, offloaded, or published, and publishing from a sharded trainer is itself a collective over the whole training mesh. Which boundary dominates cost depends on the configuration: in the measured separate-slab deployment, much of the slowdown arises from reward processing (Section 8.3).

Topology heterogeneity. A small run may time-share one GPU set between training and rollout; a larger run may separate them; a variable-duration rollout may benefit from asynchronous admission. Each choice changes memory pressure, transfer paths, synchronization frequency, and allowable policy lag.

These observations produce four requirements (Table 1). The first two follow from trajectory heterogeneity and the third from role and topology heterogeneity; the fourth specializes the third to asynchronous execution, where decoupled roles may train on work generated by an older policy.

Table 1: Design requirements and the corresponding UniRL mechanism. The right column states what must be evaluated; it is not a result.

Requirement	Mechanism	Evidence required
Preserve lineage	ID-encoded ancestry validated at construction; split, select, concat, data-parallel sharding, and reward propagation over complete root groups	Trace invariants across AR, diffusion, and cross-stage runs against an ID-joined reference
Preserve replay inputs	Typed primitives and conditions; packed AR and sparse latent replay segments; σ-schedule echo checks on external diffusion engines	Rollout–replay consistency on exercised model paths
Change execution safely	Logical roles, placement scopes, field-aware dispatch, tensor references, weight publication and residency lifecycle, and startup rejection of unvalidated combinations	Replay parity after each publication path; matched-work topology and transport ablations
Control policy age	Per-layer behavior versions, complete-group admission, lag filters, quiescent publication, mixed-version rejection, and hard boundaries	Lag distributions, throughput, and time-to-quality

Non-goals. UniRL does not erase model-specific code; it confines it to a model bundle—architecture-specific stages, conditions, and rollout adapters—and to an algorithm that defines its loss over replay. It does not automatically discover an optimal placement. The present runtime uses a driver to sequence roles and Ray actors (Moritz et al., 2018) to host them; it is not a new cluster scheduler.

3 A Trajectory-Centric Programming Model

3.1 A trajectory batch is a sequence of tree layers

The central type is

$$\mathcal{T} = [P_0, P_1, \ldots, P_L], \qquad P_i = (I_i, X_i, C_i, Z_i, R_i, A_i, V_i, M_i), \tag{2}$$

where I_i are sample identifiers, X_i decoded modality values, C_i model conditions, Z_i replay segments, R_i rewards, A_i advantages or returns, V_i the generating engine's behavior version v_b (tracked on the asynchronous and agentic paths; Section 4.3), and M_i auxiliary metadata. In code, $\mathcal{T}$ is a `Sample` and each layer P_i is represented by the batched class `Part`; we refer to P_i as a *layer* throughout the paper. The sequence encodes a forest of root-prompt trees: P_i holds every node at depth i, and edges connect adjacent layers. We call the last layer the *frontier*; trajectories grow by appending layers to it. A generated layer is typed by the sampling-parameter class that produced it and the replay segment class it carries, for example an AR layer with a `TextSegment` or a diffusion layer with a `LatentSegment`; input layers carry the prompts and observations that condition generation.

Identifiers encode ancestry. Root IDs contain no lineage separator; a branch appends a child ordinal. For every non-root layer P_i ($i > 0$) and every row identifier $s \in I_i$, its parent ID must occur in P_{i-1}:

$$\forall i > 0,\ \forall s \in I_i : \quad \text{parent}(s) \in I_{i-1}. \tag{3}$$

The constructor checks Equation 3. The `fork` operator appends k generated children per row of the frontier; `observe` appends a branch-one input from an environment; and `propagate_rewards` reduces child reward to an unscored parent.

Batch operations act on complete root groups. `Sample.split()` returns one tree-complete trajectory per root, and slicing or selection is implemented by selecting these groups rather than independent rows. Moving complete roots preserves the parent-contiguous order constructed by `fork`; independently permuting child rows need not preserve the credit computation.

Well-formed batches. Lineage operations are defined over well-formed trajectory batches: root IDs are unique, trajectories being merged have the same number of layers, and siblings follow the parent-major order produced by `fork`. The programming model establishes these properties by construction—trajectories grow only through `fork` and `observe`, and batches are sharded and selected as complete root groups—so credit assignment and reward reduction can consume each sibling group as one contiguous block. Group-advantage computation rejects batches whose siblings are not contiguous, layer concatenation requires matching decoded-value metadata across shards, and the asynchronous batch assembler admits only batches generated by a single behavior version. E0 is the gate that checks these properties at the actual experiment boundaries (Appendix D).

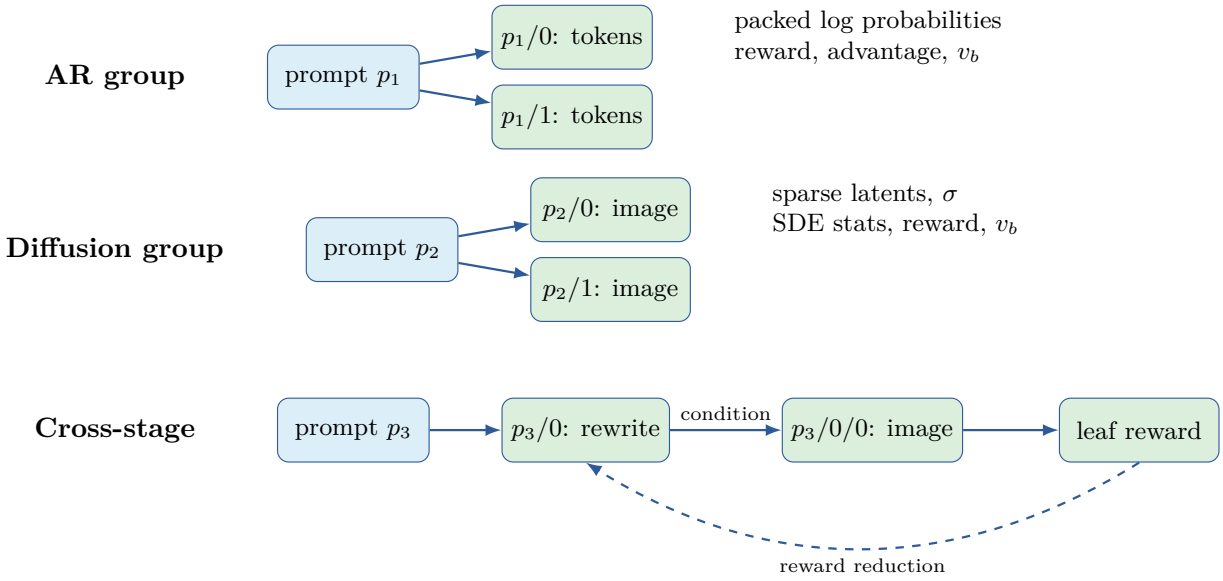


Figure 1: Three trajectory shapes represented by the same ordered-layer IR. Each generated layer retains decoded values and the replay segment required to replay its policy, together with the behavior version v_b. The cross-stage path illustrates available ancestry; E4 freezes the rewrite model and trains only diffusion.

3.2 Values, conditions, and segments are separate layers

UniRL distinguishes three kinds of leaves that are often conflated.

Primitives. Primitives are decoded or externally supplied values: text, images, videos, audio waveforms, and media references. Image pixels and variable-duration video/audio values use packed representations with per-example shapes or cumulative offsets. They are suitable for reward functions, visualization, and subsequent turns, but need not be the tensors replayed by the trainable model.

Conditions. Conditions are model-facing encodings, such as text embeddings or image latents. They allow rollout adapters and train-side stages to agree on typed inputs without prescribing one tokenizer, text encoder, or multimodal fusion architecture.

Segments. Segments retain stage-specific replay segments. `TextSegment` stores variable-length packed token fields. `LatentSegment` stores latent snapshots at selected indices, a shared sigma schedule, and optional SDE log probabilities or transition means. The sparse index map is important: a diffusion algorithm can replay only selected transitions instead of materializing an entire denoising path for every loss.

The distinction makes decoded payload lifetime independent of replay lifetime. After reward logging, trainers may drop large decoded media while retaining conditions, segments, reward, and credit for training.

3.3 A field algebra defines structural batching

Every structured batch field carries a kind: concatenated, packed variable-length, or shared (the default for an unannotated field); scalar reductions by maximum, minimum, sum, or mean are also available but are applied only when concatenating, and the current source uses only the maximum, for timing metadata. Concatenation, row selection, slicing, and repetition act recursively on concatenated and packed fields and pass shared fields through unchanged; device movement applies to every tensor regardless of kind. The same kinds define how a distributed method reconstructs its return value. For example, a data-parallel (DP) scattered `Sample` is split by root groups (the root count must be divisible by the DP size), and the DP heads' results are concatenated position-wise; another argument whose leading size differs from the split size

is replicated to every shard rather than rejected. A broadcast lifecycle method returns the list of per-worker results, and callers that expect a replicated value check agreement explicitly.

This removes hand-authored tensor slicing from each role; merging shards relies on the well-formed-batch properties above. It also creates a measurable cost: recursive structural operations and reference bookkeeping must be benchmarked in Section 8.3 rather than assumed free.

3.4 Rollout unfolds a declared trajectory tree

A rollout request is the skeleton of the tree it will become. Starting from the root layer P_0, the caller appends one generated layer per stage with `fork`; until an engine fills its tensors, we refer to such an unfilled layer in a request skeleton as a *shell*. Each shell fixes its child count, carries child identifiers that already name their parents, and holds a typed sampling-parameter object—AR or diffusion—that identifies the generative process that must fill it. The prompt-enhancement and unified-model trainers build

$$\texttt{request}(P_0).\texttt{fork}(k_1, s^{\mathrm{AR}}).\texttt{fork}(k_2, s^{\mathrm{diff}}) = [P_0,\ P_1^{\mathrm{AR}},\ P_2^{\mathrm{diff}}], \tag{4}$$

where s^{AR} and s^{diff} are the stage sampling parameters. For B root prompts, the resulting Bk_1 AR rows and Bk_1k_2 image rows are fixed before any token or latent is sampled. Every rollout engine implements one transformation over such a skeleton:

$$F_{\theta,e} : \mathcal{T}_{\mathrm{skeleton}} \longrightarrow \mathcal{T}_{\mathrm{filled}}, \tag{5}$$

where e identifies a train-side or external generation runtime. F keeps the request's layer sequence and identifiers, writes decoded values, conditions, and a replay segment into the shells it fills, and stamps its behavior version on the frontier layer it returns; when an engine or trainer fills several shells through child engines, it copies each child's behavior version onto the layer that child filled. The base contract also exposes lifecycle and weight-publication operations, but it does not standardize the engine's internal serving API.

One trajectory spans autoregressive and diffusion stages. The filled skeleton records every stage of a multi-stage generation in the trajectory itself. Within a pipeline that fills several shells, a generated shell obtains its inputs from its ancestry: `conditioning_at` walks each row's parent chain and collects the role-tagged primitives along it. In the HunyuanImage-3.0 and BAGEL text-to-text-and-image modes, one pipeline fills the AR shell from the root prompt; the image shell's lineage then surfaces both the original prompt and the generated reasoning as its conditioning, without a side table that pairs each image with the text that produced it. Each stage's replay state lands in its own layer—a `TextSegment` from an AR stage and a `LatentSegment` from a diffusion stage—so the filled tree carries what credit assignment and replay need for both stages (Section 3.5). An AR stage and a diffusion stage are thus two typed layers of one prompt-rooted tree rather than two batches joined after the fact. The representation carries this cross-stage dataflow; stage order is declared by each workflow's skeleton. The current trainers instantiate single-stage AR, single-stage diffusion, input–AR–diffusion, and multi-turn tool trajectories from the same representation.

Engine forms. The current source contains three direct forms of Equation 5 and one coordinator that composes it. The train-side adapter invokes the model pipeline directly under evaluation mode, filling every generated shell of the request. Dedicated serving adapters for SGLang, SGLang-Diffusion, vLLM, vLLM-Omni, and FastVideo translate typed trajectory inputs to their native requests and rebuild typed responses. A composed engine serially fills the AR and diffusion shells of Equation 4 with two child engines. An agentic harness composes Equation 5 over turns: it forks a policy turn, has a named engine fill it, and appends the environment observation before the next turn, returning a longer layer sequence. The claim is therefore about a common boundary, not one universal generation kernel.

3.5 Training folds the tree into per-stage updates

Training traverses the filled tree in the opposite direction. The reward role scores the frontier, which in the cross-stage layouts above is the image layer. `propagate_rewards` then reduces each scored layer onto its unscored parent layer by mean, maximum, or sum, so an upstream AR layer receives feedback from the images it conditioned. Credit is computed per trainable layer: `compute_advantages` labels each row by its

ancestor at a chosen depth—the immediate parent by default, or the root prompt when requested—and, by default, normalizes rewards within each group. For the image layer of Equation 4, the two choices compare images under one rewrite or all images under one original prompt; selecting between them is a lineage query rather than a reconstructed join (Section 8.3). Reduction and grouping consume siblings as contiguous blocks, the well-formedness property stated above. Advantages are attached on the driver before the batch is scattered, so training shards receive precomputed credit rather than re-deriving groups.

Each trainable layer then becomes a training track bound to a train stack and a stage algorithm. The trainer selects the layer by its sampling-parameter type and skips frozen layers. In the composed prompt-enhancement workflow, the AR and diffusion models have separate stacks and optimizer steps; freezing the rewrite model, as E4 does, trains only the diffusion track. When both layers belong to one backbone, as in HunyuanImage-3.0, a unified stack back-propagates the AR and image losses through one shared optimizer step.

3.6 Component interfaces for stage replay and optimization

Once credit is attached, each training track has three interfaces.

Trajectory layer. The layer supplies (C_i, Z_i, A_i) for stage i: model-facing conditions C_i, the replay segment Z_i (a `TextSegment` or a `LatentSegment`), and precomputed advantages A_i.

Stage algorithm. The algorithm defines the objective

$$(C, Z, A; \theta) \longmapsto \mathcal{L} \quad \text{and backward}, \tag{6}$$

over one shard's conditions, replay segment, and advantages. The loss depends only on (C, Z, A). An optional per-shard preparation hook may read the full layer `Part`, for example per-row metadata or critic values, and an algorithm that needs reference or exponential moving average (EMA) weights holds an explicit backend handle.

Train stack. The stack plans micro-batches, can freeze an old-policy replay anchor at the training micro-batch geometry before repeated updates, accumulates and clips gradients, normalizes global losses, and steps the optimizer.

Under this split, AR objectives replay token log probabilities from `TextSegment`s, and diffusion objectives replay selected denoising transitions from `LatentSegment`s, comparing them with rollout statistics, a pre-update replay, or a reference model. Supervised and teacher-distillation objectives do not use A; the forward-process DiffusionNFT objective consumes group advantages through the same signature.

Because models and algorithms meet only at this data boundary, they compose without pairwise adaptation. A model is integrated once, as a bundle whose stages fill and replay typed segments; an algorithm is written once, as a loss over a stage's replay of those segments. A shared algorithm can then be paired with any model whose stages produce and replay the segment fields it consumes—AR objectives with `TextSegment` producers and diffusion objectives with `LatentSegment` producers—so supporting N_{mod} models and N_{alg} shared algorithms requires $N_{\text{mod}} + N_{\text{alg}}$ integrations rather than $N_{\text{mod}} \times N_{\text{alg}}$; a model that needs a specialized objective adds it as a subclass without changing the others. Variants that keep the Flow-GRPO loss (Liu et al., 2025) (implemented by the `FlowGRPO` algorithm class) and change only the SDE transition or the step schedule, such as DanceGRPO and MixGRPO, are recipes rather than new algorithm classes.

This division is deliberately not advertised as algorithmic novelty. GRPO follows the group-relative formulation introduced for language reasoning (Shao et al., 2024); PPO follows proximal policy optimization (Schulman et al., 2017); diffusion objectives build on prior diffusion-RL methods (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025). The systems question is whether these objectives can consume the same trajectory and execution contracts without hiding their different sufficient statistics.

4 From a Program to a Distributed Execution

Figure 2 separates the feedback program from the mechanisms that execute it, where credit assignment uses either driver-side group normalization or worker-side generalized advantage estimation (GAE).

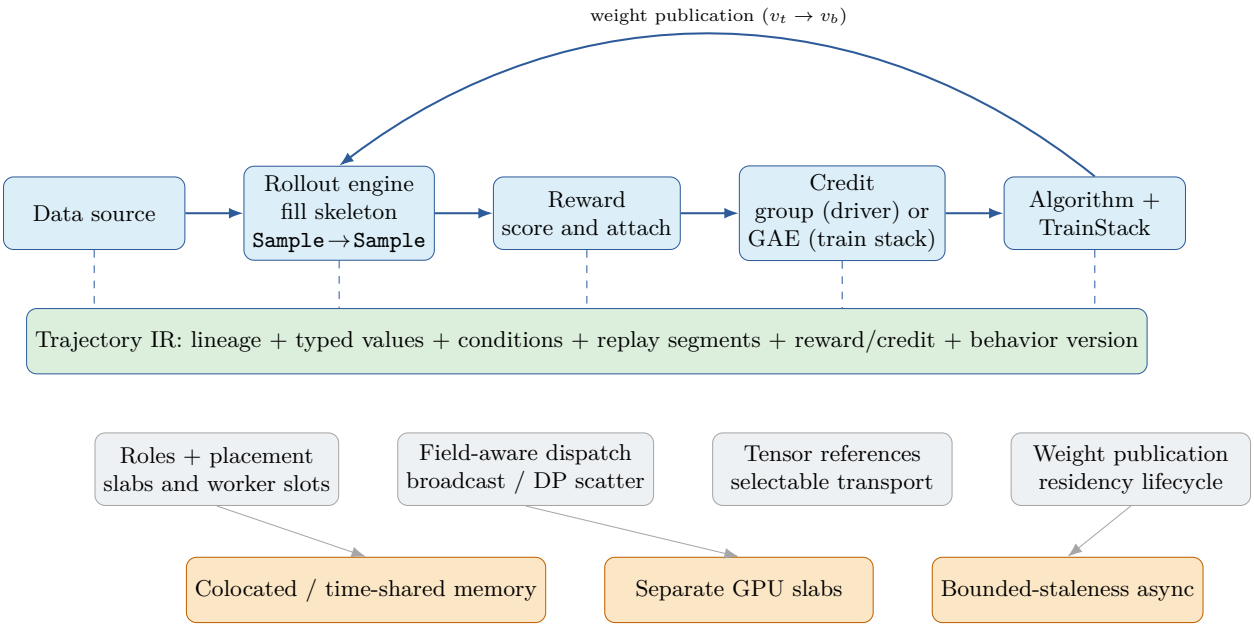


Figure 2: Source-derived UniRL architecture. The top row is the feedback program; the green bar is its stable data boundary. The lower rows map logical roles to communication and resource policies. Not every topology is valid for every engine; startup validation rejects unsupported combinations.

4.1 Logical roles, device slabs, and worker slots

A `DevicePool` creates one Ray worker per GPU initially and reserves one strict-packed placement group per node. Each worker occupies a slot on its GPU: slot 0 exists from startup, further slots are created on demand up to a per-GPU limit, and every slot reserves an equal fraction of that GPU. A top-level placement scope claims a fraction of the whole device pool from the still-unclaimed devices; a nested scope takes a fraction of its parent's devices. Roles created inside a scope either share its worker slot—enabling local sibling access and memory time-sharing—or receive isolated slots on the same devices (isolated slots require the per-GPU store transport). Separate top-level scopes form separate training, rollout, or reward slabs.

A role is a regular Python object derived from `Remote`. The handle constructs one role replica on each selected worker and derives data-, tensor-, pipeline-, sequence-, and expert-parallel rank metadata from the role configuration. Method decorators declare a dispatch mode (broadcast, scatter, or DP scatter) and which ranks execute; the dispatch mode determines how results are collected. The trainer is consequently a single-controller program, while each method may invoke multi-controller distributed training or serving inside a role. This resembles HybridFlow's separation between orchestration and distributed computation (Sheng et al., 2024); UniRL's paper claim centers instead on carrying the same typed lineage through heterogeneous generative stages.

4.2 Tensor references decouple structure from movement

In the single-controller design, every role method is a remote call issued by the driver, and its result returns to the driver before being dispatched to the next role. If trajectories carried dense tensors across these hops, latent snapshots, token log probabilities, and decoded media would pass through the driver on every call, and

the driver would have to materialize them merely to split, select, or scatter complete root groups. UniRL therefore routes control through the driver but keeps tensor data on the workers. When a role method returns, its worker stores every dense tensor leaf in a local transport and *dehydrates* it into a `TensorRef`: an ordered list of half-open row spans over opaque handles, plus shape, dtype, and device metadata. The driver receives only this dehydrated trajectory; selection and contiguous slicing construct new span views without moving tensor data. Before the next call, the transport localizes each reference for its destination worker, leaving resident spans in place and transferring the others worker-to-worker. The destination worker then *hydrates* all referenced spans in one batch and writes the tensors back into the nested object in place before invoking the method; spans whose trailing widths differ are zero-padded to a common width. Control decisions on the driver need only small fields such as rewards, which it hydrates on demand.

The transport interface has a deliberately small contract: store tensors, resolve handles, identify references, and optionally localize references for destination workers. The audited source provides (i) a colocated per-worker store, (ii) a per-GPU store with IPC and NCCL transfers, and (iii) a TransferQueue path backed by simple storage or Mooncake-compatible storage. Mooncake itself was proposed for disaggregated serving (Qin et al., 2024); here its storage interface is a transport option rather than a claimed UniRL invention.

The driver can therefore manipulate trajectory structure and metadata without materializing all GPU tensors. Whether this reduces memory and transfer cost in practice is RQ3, not an assumption.

4.3 Residency lifecycles and weight publication

Training and generation often hold different copies or formats of a policy. We distinguish two version counters: the *optimizer version* v_t, which counts committed optimizer updates on the training stack, and the *behavior version* v_b, which the rollout engine stamps onto each generated frontier layer. In every deployment, a due weight publication completes before the generation it serves, and trainers that publish through the rollout manager update v_b to match v_t upon publication (Section 4.5); the remaining cases are described below. How weight publication interleaves with memory residency depends on whether the two roles share devices.

Colocated slabs. When training and rollout time-share one GPU set, the engine is put to sleep—releasing its weight and cache memory—between generations and woken before the next one, and training state may be offloaded to host memory while the engine is awake. Every colocated order keeps two invariants: a due publication completes before generation, and memory is released before the role that needs it is brought up, so the engine sleeps before offloaded training state is restored. The default order wakes the engine, publishes weights if due, offloads training state if enabled, and generates; afterwards it sleeps the engine and restores the training state. The AR and agentic trainers follow this order, and the AR trainer adapts it to engine capabilities: an engine that can restore its weight memory separately from its cache receives published weights before allocating the rest, and an engine pinned to one GPU of the slab receives weights that are extracted before training state is offloaded and published after the engine wakes (Appendix B). The diffusion trainers express residency as one resident-or-offloaded choice per role (training, rollout, reward): they offload every non-resident role before waking the engine, publish weights into the awake engine, sleep it after generation unless the rollout role is resident, and restore an offloaded trainer only before its optimizer step. The prompt-enhancement trainer follows the default order, and the unified-model trainer with separate AR and diffusion engines uses the same extract-then-publish order. A train-side engine reads live model objects, so training state is never offloaded for it.

Separate slabs. When training and rollout occupy separate GPU slabs, generation needs no memory handoff:

1. at startup, connect the training side to the rollout workers through a one-time handshake;
2. at each publication point, publish parameters across slabs, under the admission discipline of Section 4.5 for asynchronous trainers;
3. generate and stamp the frontier layer with the engine's current behavior version v_b.

Generation never offloads training state to make room for the engine. The synchronous diffusion trainer sleeps the engine between rollouts unless the rollout role is resident and, when the training role is not resident, offloads training state while a reward role shares the training slab. The asynchronous trainers keep the engine awake; the asynchronous diffusion trainer also requires a resident reward role.

Weight-publication implementations cover in-process and cross-slab Low-Rank Adaptation (LoRA) adapter publication as well as full-weight publication via serialized tensors, CUDA IPC (ZMQ or vLLM's native weight-transfer engine), SGLang checkpoint-engine IPC, NCCL broadcast, and a checkpoint-file handoff. The base engine can also receive per-track updates for composed models.

The stamped behavior version v_b tracks optimizer-version provenance on the asynchronous and agentic paths, which set $v_b = v_t$ at each publication. The synchronous trainers do not advance v_b, so their frontiers keep version zero, except when publishing through vLLM's native weight-transfer engine, which increments v_b on each received publication.

The source rejects combinations whose lifecycle has not been validated, such as a train-side rollout on a physically separate slab or selected EMA/offload combinations. This fail-closed policy is important because a partially awakened engine or silently stale adapter can produce plausible but invalid training data. Section 8 therefore includes replay-consistency and failure-injection checks alongside throughput.

4.4 Synchronous execution

The synchronous AR trainer performs rollout, reward, credit assignment, replay/update, optional evaluation, and checkpointing in order. The diffusion trainer can accumulate multiple rollouts into one optimizer window and can place reward on the train slab or a third slab, or score micro-batches inside the colocated rollout window so that reward overlaps generation. Topology is a placement choice made through the same scopes for every modality. The synchronous AR trainer builds one colocated slab, optionally pinning the engine to one of its GPUs, and the synchronous diffusion trainer additionally builds separate training and rollout slabs. Separate-slab AR without overlap is the asynchronous trainer with one prompt batch in flight, publication after every batch, and zero allowed lag (Section 4.5): it admits the next batch only after publishing the weights produced from the previous one, so generation and training strictly alternate. In every case, topology changes resource ownership, residency, and the publication path, but not the trajectory boundary: the same `Sample` passes from rollout through reward and credit to the train stack. This makes a controlled topology experiment possible: hold model, prompts, group size, reward, number of replayed steps or tokens, and optimizer updates constant while changing only engine or placement. The publication path and training DP width change with the placement and are reported as part of it.

4.5 Bounded-staleness execution

Asynchrony changes correctness conditions, not just control flow. UniRL's asynchronous AR and diffusion trainers run on separate training and rollout slabs and share one manager. The driver submits whole prompt batches only while they fit the in-flight capacity, the freshness horizon implied by the staleness allowance, and the batches remaining before the next evaluation, checkpoint, or final boundary. A background dispatcher then fills every engine slot up to a configured per-worker capacity. Completed trajectories are grouped by root; only complete groups enter the ready buffer. Every generated layer must carry a behavior version v_b.

Let v_t be the current optimizer version, which counts committed optimizer steps, and u the configured number of optimizer updates per consumed rollout batch. The manager exposes lag in update units,

$$\ell = v_t - v_b \leq ub, \tag{7}$$

where b is the staleness allowance in rollout-batch units (configurable for AR, fixed to the publication interval minus one for diffusion). The filter measures each trajectory by its oldest generated layer and discards whole root groups beyond that budget, including incomplete roots held across a publication. Because a skipped non-finite step does not advance v_t, admitted work can be more than b rollout batches old. Before publishing optimizer version v_t, the manager pauses admission, drains or suspends work, completes any started or batch-aligning prefix, checks that no publication splits a training batch across behavior versions, publishes weights,

updates the engine's behavior version to v_t, and then resubmits eligible pristine carry. Evaluation and checkpointing require a fully empty manager.

This discipline differs from claiming fully asynchronous optimization. Weight publication remains a barrier. The asynchronous diffusion trainer admits one prompt batch at a time and requires each training batch to come from a single generation request, so its next generation overlaps only the optimizer step; the AR trainer may keep several batches in flight and refills before reward scoring. The evaluation must report the actual lag histogram, dropped work, and barrier time; a nominal "async" label is insufficient.

5 Instantiations of the Common Contracts

The following paths show which information crosses the common boundary and which computations remain specialized. They illustrate the contracts at the design level, as implemented in the audited source, rather than inventory the system's capabilities. Section 8 exercises only the group-relative AR objectives, Flow-GRPO, and the frozen-rewrite cross-stage path; the GAE, DiffusionNFT, rewrite-training, and agentic paths described below have stock recipes but no reported training runs.

5.1 Autoregressive reasoning

An input layer carries prompt text and metadata. Forking by group size creates generated shells with sibling IDs. An AR engine fills packed token segments and decoded text, and a reward role attaches scalar or component rewards. Group-normalized credit is computed on the driver before the batch is scattered; token-level generalized advantage estimation (GAE) is instead computed per trajectory on the training workers, after the critic anchor is frozen. The train stack replays the same segment and applies the selected objective. Optional token-length balancing permutes individual generated rows of the training layer before the train stack scatters them, after driver-side group credit but before per-trajectory GAE, which does not depend on row order. It keeps each row's identifier and any attached advantage but abandons sibling contiguity, so no grouping operation may follow it; the E3-AR UniRL recipe enables it.

5.2 Diffusion and flow generation

A diffusion request forks prompts and records sampling parameters, selected SDE indices, and an initial-noise recipe; the hosting engine pins the sigma schedule onto the shell before generation. The engine returns decoded media plus a latent segment whose sparse trajectory fields are sufficient for the selected update. Flow-GRPO-style objectives replay recorded steps and compare new transition log probabilities against a frozen rollout or replay anchor (Liu et al., 2025). Forward-process objectives such as DiffusionNFT use clean generated samples rather than the same reverse-trajectory likelihood path (Zheng et al., 2025). The common train stack handles micro-batching and optimizer state; the algorithm remains responsible for these incompatible estimators.

5.3 Cross-stage and environment-interacting trajectories

In a prompt-enhancement path, one root produces one or more AR rewrites, and every rewrite produces one or more images (Equation 4). The image reward is attached to the leaf; when the rewrite stage trains, parent-major reduction supplies its feedback. The primary E4 case freezes that stage and uses ancestry to select the diffusion advantage groups. The prompt-enhancement workflow needs this ancestry to survive rollout scheduling and sharding; flattening it into unrelated rows would require rebuilding that correspondence out of band. In a tool-interacting path, each trajectory is a single-branch chain: the policy emits an action-like text layer, the environment appends an observation layer, and the process repeats until termination or suspension. The agentic trainer keeps one `Sample` per trajectory, forms a group from repeated copies of the same root rather than forked siblings, computes group advantages over those trajectories, and trains on their flattened generated turns, each inheriting its trajectory's advantage, so lineage there conditions each turn within a trajectory rather than defining groups or shards.

E4 checks these groups against an explicit ID-based reference on synthetic fixtures, leaving the recorded-rollout check open, and then studies the two grouping objectives in training. Before turning to the main E0–E6 evaluation program in Section 8, Sections 6 and 7 present two targeted case studies that examine how the reward-serving and rollout-engine contracts behave on instruction-guided image editing.

6 Case Study I: Reward Serving and Generation–Reward Overlap

Reward evaluation is the one role in the feedback loop whose computational cost is independent of the policy architecture. A lightweight contrastive preference model scores a batch in well under a second, whereas a vision–language judge with tens of billions of parameters can take as long as the generation it evaluates, requires a separate software environment from the trainer, and may not fit alongside the policy on the same accelerator. UniRL therefore fixes the reward interface at the trajectory boundary while treating placement, environment isolation, and scheduling relative to generation as orthogonal execution choices.

Source scope. This case study examines UniRL commit `651490e` (2026-09-30), which postdates the v0.2.0 design-audit commit analyzed in Sections 3–5, together with the shared-GPU mechanism proposed in UniRL pull request #492 (commit `8bb4119`). The measurements in Section 6.4 were collected as a standalone systems case study outside the E0–E6 evaluation program; their per-rollout records and table generators are archived in `artifacts/reported_results/2026-09-30-reward-service/`.

6.1 Scoring Contract and Failure Semantics

A trainer invokes a single method, `score_and_attach`, on the reward role. The call is scattered across tree-complete shards like any other distributed role method (Section 4); on each shard it constructs a typed request from the trajectory, queries a configured backend for scores, and returns an updated trajectory whose frontier layer carries a scalar reward and named component rewards per row. The request contains the frontier layer's decoded values, non-text values from earlier turns as conditioning (for example, the source image in an image-editing task), both the original root prompt and the rewritten generation prompt, and the sample and group identifiers. Each backend specifies which prompt field it scores against; when a prompt-rewrite stage precedes generation, the two prompts differ, and this configuration determines whether the reward measures fidelity to the original user request or to the rewritten prompt.

To prevent silent propagation of invalid credit, the reward role enforces strict validation on scored outputs. It rejects a frontier layer that already carries rewards, raises an exception when any row fails rather than substituting a default zero (remote backends treat any non-finite score as a failure), and assigns zero reward or a configured length penalty to free-length autoregressive traces that hit the token limit without terminating. For audio–video generation, it also rejects training reward configurations that assign zero weight to prompt-to-video alignment, because optimizing an audio–video agreement score alone permits degenerate solutions that collapse both streams; such scores remain permitted in evaluation-only mode. Because group-normalized advantage estimation couples every sibling under a prompt, a single corrupted reward would contaminate the entire group; failing closed prevents corrupted updates from entering the optimizer.

Table 2 summarizes the scorers that implement this contract: 19 in-process backends and 13 scorers available in the standalone reward service's registry. These span contrastive and preference models such as CLIP, PickScore, HPS, and ImageReward (Radford et al., 2021; Kirstain et al., 2023; Wu et al., 2023; Ma et al., 2025b; Xu et al., 2023), vision–language judges for composition, rendered text, and instruction-guided editing (Kamath et al., 2025; Wu et al., 2025; Luo et al., 2025), video and audio–video alignment models, and text verifiers for reasoning tasks. A remote backend can query a panel of scorers for each row and aggregate the results via a weighted sum, mean, minimum, or maximum while preserving individual component scores for logging. Inclusion in Table 2 indicates that the adapter code path exists in the repository; it does not imply that every listed scorer has been benchmarked in end-to-end training.

Table 2: Reward implementations at UniRL commit `651490e`, grouped by scored output modality. Names denote registry keys. *Present in the registry but disabled in the default service configuration.

Output scored	In-process backends	Reward-service scorers
Image	`clip`, `pickscore`, `hpsv2`, `hpsv3`, `hpsv3pp`, `image_reward`, `ocr`, `geneval2`	`clip`, `pickscore`, `imagereward`, `hpsv2`, `hpsv3`, `unified_reward`, `geneval2`, `geneval`*, `ocr`, `wise`
Image editing	–	`editreward`, `editscore`
Video	`videopickscore`, `videoclipdelta`, `videoalign`, `video`	`videoalign`*
Audio	`clap`	–
Audio–video	`imagebind`, `t2av_composite`	–
Text	`math_verify`, `mc_exact_match`, `llm_judge`	–
Routing	`per_domain`	–

6.2 Scorer Placement and Isolation Modes

The same scoring contract can be served from four execution modes that trade off communication overhead against memory and dependency isolation.

In-process on policy workers. An in-process backend loads the scorer directly inside the worker that hosts the training and rollout roles. Its device memory is governed by the per-role residency policy described in Section 4.3: each role (trainer, rollout engine, and reward) is configured as either resident or offloaded when idle, and the residency planner issues only the state transitions needed between phases. When reward evaluation immediately follows generation, the reward role executes while the trainer remains offloaded, avoiding redundant host–device transfers.

On a separate reward slab. A placement scope can allocate a fraction of the device pool as a separate GPU slab dedicated to the reward role. Scoring calls are scattered across the workers of that slab and receive decoded media via tensor references, isolating heavier scorers from policy memory pressure while keeping the workflow within a single distributed program.

In a managed child process. Scorers that depend on dedicated serving engines or conflicting library versions often cannot be imported into the trainer's Python environment. A managed child backend spawns one subprocess per reward worker using a separate Python interpreter, binds it to a local loopback HTTP socket and that worker's assigned GPU, and communicates over a strict request–response protocol. Requests are transmitted in bounded chunks independent of the data-parallel shard size. Every item carries its sample ID, group ID, source rank, behavior version, and scorer version, all of which the child process must echo in its response. An idempotency key derived from these identifiers and a payload fingerprint allows retried requests to return cached scores without re-evaluating the model. The child process serializes scoring requests with its memory-lifecycle transitions and rejects non-finite outputs. The parent worker can either keep the child's model resident on the GPU or wake it only for the duration of each scoring call. A managed child process can host a full vLLM engine (Kwon et al., 2023): for the EditScore judge integrated in UniRL pull request #348, offloading to host memory takes 5.1 s (leaving 1.0 GB of 47 GB allocated on the device), waking takes 0.3 s, and returned scores are bit-identical across offload–wake cycles. In the audited implementation, managed child processes support image and image-editing requests.

In a standalone service. A standalone reward service places an HTTP gateway in front of Ray actor groups (Moritz et al., 2018). Each configured scorer runs as a pool of replica actors that reserve dedicated GPUs and initialize their own package dependencies on startup. The gateway decodes each incoming batch once, groups items by requested scorer, dispatches requests across replicas in round-robin order, and awaits each scorer under an independent timeout. Scorer failures are reported per item and per scorer alongside successful results so that a single failed backend does not abort the entire batch; the trainer client treats any missing score as a row failure. Because actor groups within the Ray cluster share the base Python

interpreter and PyTorch runtime, conflicts at the interpreter level require either the managed child backend or an external single-scorer HTTP server launched on a separate node, which is the deployment used for the remote judges in Section 6.4.

6.3 Overlapping Reward Evaluation with Generation

In the default synchronous diffusion loop, all data-parallel ranks finish generating their full shards before the driver dispatches reward scoring. When scoring uses a remote judge, this barrier introduces two sources of latency: the training GPUs remain idle while the remote judge processes the batch, and requests from all data-parallel ranks arrive at the judge simultaneously, creating a burst queue.

An optional worker-side role, `RewardStack`, pipelines generation and scoring at the micro-batch level without altering the outer trajectory contract. Colocated on the same worker as the rollout engine and reward client, `RewardStack` partitions its shard's frontier layer into micro-batches of a configured row count and invokes generation followed by scoring on each micro-batch. Scored micro-batches are concatenated in their original row order so that sibling contiguity and group credit assignment remain unchanged, and the driver bypasses its separate post-generation reward phase. A micro-batch may be as small as a single row; if the underlying rollout engine requires an entire sibling group per request, the micro-batch boundary is rounded up to avoid splitting a group (Section 7.3 removes this constraint for the vLLM-Omni BAGEL adapter). When `overlap` is enabled, scoring requests are submitted in order to a single background thread per worker so that generation of the next micro-batch proceeds immediately; any scoring exception is surfaced after the subsequent micro-batch completes generation.

The `RewardStack` role enforces fail-closed validation on unsupported topologies. Because it requires the rollout engine and reward client to reside on the same worker, startup validation rejects configurations that use a separate rollout slab, a separate reward slab, a non-resident reward role during generation, or sequence- or tensor-parallel rollout groups in which multiple ranks collaborate on a single shard. Overlapped execution also imposes one reproducibility constraint: if an in-process scorer consumes numbers from the global PyTorch random number generator, background scoring threads would interleave with the diffusion sampler's noise draws and make rollouts non-deterministic; such scorers must therefore use the serial micro-batch schedule. Asynchronous trainers achieve overlap through a different mechanism: the asynchronous AR trainer admits the next generation batch before scoring the completed batch, while an open UniRL pull request extends the asynchronous diffusion manager to release rollout capacity while remote reward requests are in flight.

6.4 Empirical Evaluation of Micro-Batched and Overlapped Schedules

Experimental setup. We evaluate the scheduling modes on a single node of eight NVIDIA H20 GPUs fine-tuning BAGEL (Deng et al., 2025) for instruction-guided image editing with Low-Rank Adaptation (LoRA), using eight data-parallel vLLM-Omni rollout engines. Each rollout processes 32 prompts with eight image edits per prompt (32 rows per rank), which a micro-batch size of eight partitions into four micro-batches per rank. An EditScore vision–language judge (Luo et al., 2025) hosted on a second node scores each source–edit pair over HTTP in either an 8B or a 72B model variant, each served by a single tensor-parallel vLLM instance behind the standalone single-scorer server. Each run executes two to three rollouts; the first rollout is excluded as warm-up and steady-state rollouts are pooled across repeated sessions on two allocations of identical hardware. Phase durations are recorded by the driver's monotonic phase timers.

Because these runs were collected prior to the conditioning-injection fix analyzed in Section 7.1, the image-editing wall-clock durations in Table 3 and Figure 3 are rescaled estimates reflecting the restored conditioning path. Specifically, generation durations are scaled by the ratio between the fixed and unfixed conditioning paths measured at 16 rows per rank in Table 7, under the assumption that the same factor applies at 32 rows per rank across all micro-batches. Reward-side durations—judge service time, queueing delay, and non-overlapped reward wait time—are retained directly from the measurements, as the judge evaluates the same number of 512×512 image pairs at identical request batch sizes. When the judge has idle capacity during generation, the full generation time reduction carries over to end-to-end rollout time; when the judge is saturated, only the earlier arrival of the first micro-batch reduces total wall-clock time. Both the raw

measurements and the rescaling model are archived in the artifact bundle. Because the rollout counts are small and cover a single model and workload shape, these results characterize the mechanism's behavior and approximate magnitude rather than a multi-seed performance distribution. In 13 of the runs, worker-side tracing additionally logged the start and completion timestamps of every generated micro-batch and HTTP reward call, while the judge server logged request arrival and completion times. Judge service times, total judge busy time, queueing delays, and per-pair scoring costs reported below are taken directly from these traces; arrival timestamps and overlap windows that depend on generation speed follow the rescaled estimate and are explicitly labeled as estimated.

Table 3: Generation plus reward wall-clock time per rollout across three scheduling modes (mean and min–max range over steady-state rollouts). Image-editing rows pair BAGEL with a remote EditScore judge; the bottom block pairs BAGEL text-to-image generation with an in-process PickScore backend. Overlapped rows pool the merged implementation (which queues all completed micro-batches) with an earlier prototype (which kept at most one request in flight); the artifact bundle reports both separately. †Estimated for the restored conditioning path of Section 7.1 from measured reward-side traces; rollout counts reflect the underlying traced runs.

Schedule	Rollouts	Generate + reward (s)	Range (s)	vs. whole shard
EditScore 8B, remote node				
Whole shard, driver-dispatched reward†	4	102.0	101.1–103.0	–
Micro-batches, serial†	5	86.9	86.3–87.3	−14.8%
Micro-batches, overlapped†	7	82.3	82.0–82.7	−19.4%
EditScore 72B, remote node				
Whole shard, driver-dispatched reward†	2	155.0	154.5–155.4	–
Micro-batches, serial†	4	138.8	138.2–139.8	−10.4%
Micro-batches, overlapped†	6	140.1	138.1–142.9	−9.6%
PickScore, in-process				
Whole shard, driver-dispatched reward	2	56.0	56.0–56.0	–
Micro-batches, serial	2	52.4	52.4–52.5	−6.3%
Micro-batches, overlapped	2	52.3	52.3–52.3	−6.5%

Regime 1: Judge with spare capacity. With the 8B EditScore judge, serial micro-batching reduces combined generation and reward time from 102.0 s to 86.9 s (14.8%), and overlapping generation with scoring further reduces it to 82.3 s (19.4%). Generation itself requires approximately 73 s across all three schedules (Figure 3); the schedules differ in how much reward latency remains exposed after generation completes. Under the whole-shard baseline, all eight ranks submit their 32-pair requests simultaneously at the end of generation; the server processes them sequentially at 3.08 s per request, so the last rank waits 24.6 s for a score that takes the judge 3.08 s to compute. Under serial micro-batching, the eight ranks contend simultaneously only on their first micro-batch, where the maximum wait is 9.6 s. Those staggered completion times desynchronize the ranks on subsequent micro-batches, allowing later requests to reach an idle judge and return in a median of 1.3 s. Under overlapped micro-batching, generation proceeds without waiting for reward responses, so the ranks remain synchronized and contend on every micro-batch; however, those queueing delays occur concurrently with the generation of subsequent micro-batches, leaving only the final micro-batch's scoring latency exposed after generation finishes. Total judge busy time is 37.7 s per rollout, which fits within the estimated 54.8 s window between the arrival of the first micro-batch and the completion of generation.

Regime 2: Saturated judge. With the 72B EditScore judge, serial micro-batching reduces combined generation and reward time from 155.0 s to 138.8 s (10.4%), while overlapped micro-batching yields 140.1 s (range 138.1–142.9 s versus 138.2–139.8 s for serial), showing no further improvement. In both micro-batched schedules, the 72B judge remains continuously busy from the arrival of the first micro-batch (an estimated 18.2 s into the rollout) until the final request completes. Consequently, total rollout plus reward time is lower-bounded by the first request's arrival time plus the judge's total service work, regardless of whether

subsequent requests are issued serially or overlapped. Total judge service work per traced rollout ranged from 119.7 to 121.4 s under the serial schedule and from 119.5 to 124.5 s under the overlapped schedule, with rollout-to-rollout variation of up to 3 s within the same arm; the small difference between the serial and overlapped means in Table 3 is within this run-to-run variation. Once the judge is fully utilized throughout the overlap window, further wall-clock reductions require either additional judge replicas or lower per-request scoring cost.

Conditions for judge saturation. Whether a deployment enters the saturated regime depends on the ratio of total reward service work to the generation overlap window rather than on model size alone. Pairing a 72B judge with a 7B-active-parameter policy and serving all eight training ranks from a single judge replica intentionally stresses the reward bottleneck. Even at the lower per-pair cost of whole-shard 32-pair requests, a single 72B judge replica requires 76.9 s of work per rollout, which exceeds the estimated 54.7 s window between the first micro-batch arrival and the end of generation. Moreover, the standalone single-scorer server processes one HTTP request at a time without cross-request batching, so concurrent eight-pair requests from different ranks queue serially rather than merging into a larger forward pass. On the 72B judge, reducing the request size from 32 pairs to eight increases the per-pair service cost from 0.30 s to 0.47 s, raising total judge work per rollout from 76.9 s to 120.4 s. Operating the 72B judge with headroom at this batch shape would require multiple judge replicas, while cross-request continuous batching or prefill–decode separation would reduce the number of replicas needed. We include the single-replica 72B configuration as a boundary stress test of the overlap schedule rather than a recommended production provisioning.

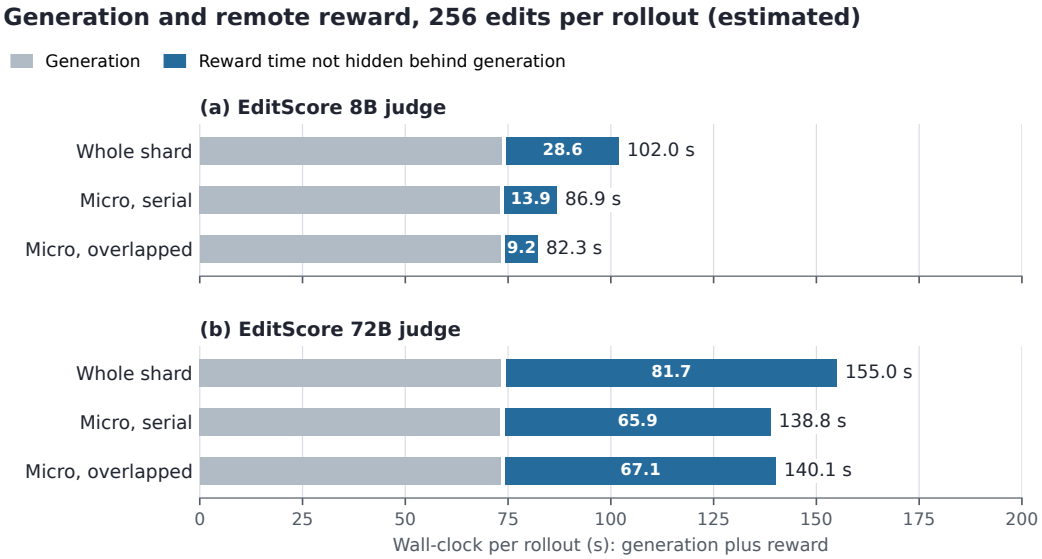


Figure 3: Breakdown of rollout wall-clock time for the two remote EditScore judges in Table 3 (estimated as described in the table caption). Generation duration is constant across schedules; the schedules differ in the portion of reward evaluation time exposed outside generation.

Batch-size overhead of micro-batched requests. Splitting a shard into smaller requests increases total judge compute time when the server does not batch across concurrent requests. On the 72B judge, an eight-pair request requires 3.76 s whereas a 32-pair request requires 9.61 s, increasing total judge work per rollout from 76.9 s (whole shard) to 120.4 s (micro-batched); on the 8B judge, the corresponding totals are 24.6 s and 38.5 s (with per-pair costs of 0.47 s versus 0.30 s on 72B and 0.150 s versus 0.096 s on 8B). Micro-batched scheduling reduces wall-clock time despite this overhead because judge evaluation begins one quarter of the way through generation rather than after all generation completes. When requests are split into smaller chunks *without* interleaving them with generation, only the batch-size penalty remains: in an ablation at 16 rows per rank, splitting the post-generation whole-shard call into two eight-row requests increased total time from 52.2 s to 57.2 s on the 8B judge and from 86.1 s to 100.7 s on the 72B judge. Adding cross-request

continuous batching or additional judge replicas would eliminate the small-batch penalty while preserving the early-start advantage of micro-batching.

In-process scoring baseline. When PickScore runs in-process on the policy workers, reward evaluation accounts for roughly 1 s of compute per rollout, and `RewardStack` reduces combined generation and reward time from 56.0 s to 52.4 s primarily by eliminating the separate driver-level dispatch hop. Because in-process scoring is already negligible relative to generation, background thread overlap provides no additional reduction (52.4 s versus 52.3 s).

Interaction with training-phase memory collection. Executing multiple generation and scoring micro-batches within a single rollout leaves uncollected Python objects and cached allocations in the worker heap. When the subsequent training phase began without an explicit garbage-collection and cache-flush pass at the rollout boundary, training step time increased from 178.1 s to 192.2 s in the 8B-judge runs and from 54.5 s to 58.0 s in the in-process PickScore runs. The merged `RewardStack` implementation therefore triggers a cleanup pass once the scored shard leaves the worker. Because the training phase replays against the trainer's own conditioning in both configurations, these training durations are direct measurements rather than rescaled estimates, illustrating why rollout scheduling changes must be evaluated across the full iteration.

6.5 GPU Sharing Across Lightweight Scorers via MPS

By default, the standalone reward service allocates whole GPUs to each scorer actor. While this provides hardware fault isolation, a panel of lightweight preference models leaves a high-memory accelerator largely underutilized. Although Ray supports fractional GPU allocation to colocate multiple actors on a single device, standard CUDA contexts from separate processes are time-sliced by the driver. NVIDIA Multi-Process Service (MPS) allows kernels from distinct processes to execute concurrently on the same GPU (NVIDIA Corporation), preserving per-scorer process and dependency isolation while overlapping their compute kernels.

UniRL pull request #492 introduces an opt-in MPS runtime mode for the standalone reward service. The service can either manage the node's MPS control daemon directly—starting it before any scorer actor initializes CUDA and stopping it after all actors exit—or attach to an externally managed daemon. Each scorer configuration may specify an active-thread percentage and a device-memory allocation cap, which are exported to the actor's environment prior to CUDA initialization. Startup validation rejects MPS configurations that combine device sharing with tensor-parallel scorers, unqualified scorer backends, or half-precision weights running below a 100% active-thread cap. During shutdown, the service halts request admission, drains in-flight GPU work, and terminates actors before stopping the MPS daemon. Because the MPS memory cap bounds per-client allocations without providing hardware memory protection, all actors sharing a GPU belong to a single fault domain, which is recorded in the service's health metadata.

Table 4: Throughput gain (%) of NVIDIA MPS over fractional GPU colocation without MPS for two or three replicas of a single scorer on one NVIDIA H20 GPU, as reported in UniRL pull request #492; B_s denotes the scorer batch size. Dashes indicate batch sizes at which the scorer implementation does not execute a single batched forward pass.

	Two copies			Three copies		
Scorer	B_s=1	B_s=4	B_s=8	B_s=1	B_s=4	B_s=8
CLIP	+11.2	+20.0	+14.8	+16.5	+24.4	+19.2
HPSv2	+17.4	–	–	+48.4	–	–
ImageReward	+24.0	–	–	+40.8	–	–
HPSv3	+20.6	+11.2	–	+22.6	+12.8	–
GOT-OCR	−2.8	–	–	−3.2	–	–

Reported qualification results. Table 4 summarizes the qualification benchmarks reported in UniRL pull request #492 on NVIDIA H20 GPUs (these measurements were not independently rerun for this report). For heterogeneous panels colocated on a single H20 GPU, the pull request reports a median throughput gain of 24.9% for three lightweight scorers and 9.6% for two, smaller gains when a heavier vision–language scorer dominates device time, and no gain for an autoregressive OCR scorer. On the deterministic test fixture, output scores were bit-identical with and without MPS enabled. Profiling in the pull request indicates that a single CLIP replica at B_s=1 utilizes roughly half of the device's execution time, allowing MPS to fill the inter-kernel gaps with work from colocated replicas; once a single scorer saturates the GPU, MPS provides no additional throughput.

Numerical correctness boundary. The same qualification identified a numerical defect on the tested H20 driver stack when restricting the MPS active-thread percentage below 100%: half-precision matrix multiplications produced non-finite outputs in CLIP and finite but numerically incorrect values in PickScore. Consequently, the runtime rejects half-precision scorers configured with less than a 100% active-thread share. A separate review on a different GPU model and driver version did not reproduce the numerical corruption and observed negligible colocation overhead even without MPS because the workload was bottlenecked by CPU preprocessing and HTTP serialization. Both the throughput benefit and the sub-100% thread-cap hazard are therefore hardware- and driver-dependent. Only CLIP and PickScore at a 100% active-thread share are marked qualified in UniRL pull request #492; vision–language judges, long-horizon stability runs, and sharing GPUs between reward actors and policy workers remain outside its qualified scope.

7 Case Study II: Rollout Fidelity and Request Shaping

Whereas Section 6 examined how generated trajectories are scored, this case study examines the boundary between the trajectory IR and an external rollout engine. Two properties at the engine boundary govern whether optimization updates match the scored behavior and how efficiently rollouts execute: first, the rollout engine and the training backend must condition on identical context representations; second, requests sent to the engine should pack sibling samples under a shared prompt so that prompt conditioning is encoded once per group rather than once per sample. Because a rollout adapter returns a structurally well-formed `Sample` regardless of how the engine internally constructs its conditioning or batches requests, verifying these properties requires explicit checks and request-shaping contracts across the engine boundary.

Source scope. This case study examines four stacked changes to the vLLM-Omni BAGEL rollout path proposed in UniRL pull requests #547–#550 (open at the time of writing), affecting both text-to-image (`t2i`) and instruction-guided image-to-image editing (`it2i`) workloads. All measurements below were collected on a single node of eight NVIDIA H20 GPUs, with per-run records and table generators archived under `artifacts/reported_results/2026-10-04-rollout-fidelity/`. The dynamic load-balancing prototype evaluated in Section 7.5 was measured on an experimental branch without an open pull request.

7.1 Conditioning Mismatch Undetected by the Importance Ratio

In instruction-guided image editing (`it2i`), the policy conditions on a source image alongside the edit instruction. To guarantee rollout–replay consistency, UniRL constructs this conditioning on the worker—applying its own image transforms and vision encoder—and injects the resulting key–value contexts directly into the vLLM-Omni serving engine so that the trainer replays against identical contexts during optimization. Following an upstream serving-engine migration, however, three helper functions in the BAGEL pipeline continued to read the injected conditioning field from the outer request-batch wrapper after the caller had already unwrapped it to an individual request object. Because the looked-up attribute was absent on the unwrapped object, the injected contexts were silently ignored, and the serving engine fell back to its internal image-to-image preprocessing path—using a different image transform at a fixed resolution, a sampled rather than mean latent encoding, and an output canvas sized to the source image rather than the recipe target.

Despite this bug, training completed without raising an error: every image-editing rollout generated samples under the engine's fallback conditioning (5,938 context tokens) and replayed them under the trainer's condi-

tioning (2,263 context tokens at the same 512×512 source resolution). Notably, standard importance-ratio monitoring did not flag the discrepancy. Before the first optimizer update of a rollout, the current policy θ equals the behavior policy π_v, so the importance ratio ρ against the rollout behavior anchor should equal 1 up to floating-point noise. In practice, the observed ratio remained close to 1 despite the mismatch: the maximum first-update deviation $|\rho - 1|$ was 5.0×10^{-5} with the conditioning bug versus 3.4×10^{-5} after the fix, well within the 5.4×10^{-5} deviation observed in healthy image-editing runs (Table 5). This insensitivity arises from how denoising transition log probabilities are reduced. Both the rollout engine and the trainer compute the Gaussian log-density of each denoising transition by averaging over all latent elements rather than summing across them; consequently, a perturbation to the predicted velocity mean—even one large enough to alter the generated image—is divided by the number of latent dimensions (tens of thousands of elements at 512×512 resolution) before entering the log-probability ratio. Under element-mean reduction, the importance ratio cannot distinguish a structural conditioning mismatch from benign numerical noise. Moreover, when a recipe uses the trainer-recomputed replay anchor (the default in several diffusion configurations), the baseline log probabilities are recomputed on the trainer itself, making the first-update ratio identically 1 by construction.

To detect such discrepancies directly, UniRL adds a structural token-layout check across the rollout–replay boundary (UniRL pull request #548). The engine hook that injects initial noise now also records three integers and the output canvas dimensions—the generation context token length, its positional offset, and the rendered height and width—which are returned through the capture channel into the trajectory's model conditions C_i and compared row-by-row against the context reconstructed by the trainer during replay. Any discrepancy raises an exception on the first replay pass and logs both layouts. Because the trainer already constructs the replay context, the check adds zero tensor-compute overhead; conditioning paths that carry pre- materialized contexts bypass the check. Table 5 summarizes the validation probes caught by this check. The importance-ratio check is retained as an optional secondary diagnostic for detecting weight-publication desynchronization or mismatched LoRA adapters, where the model weights themselves differ between rollout and replay.

Table 5: Validation probes for the structural token-layout check across text-to-image (`t2i`) and image-to-image editing (`it2i`) paths. "Engine" and "Trainer" columns report the generation-context token counts on each side. The final column reports the maximum first-update importance-ratio deviation $|\rho - 1|$ in the same run.

Probe	Path	Engine	Trainer	Layout check	$\lvert\rho - 1\rvert$
Main before the injection fix	it2i	5,938	2,263	raises at the first replay	5.0×10^{-5}
Trainer prompt lengthened by three tokens	t2i	63	66	raises at the first replay	–
Trainer source image halved in width	it2i	2,268	1,161	raises at the first replay	–
After the injection fix	it2i	2,263	2,263	passes	3.4×10^{-5}
Healthy run (rollout anchor)	t2i	equal	equal	passes	8.6×10^{-6}
Healthy run (rollout anchor)	it2i	equal	equal	passes	5.4×10^{-5}
Healthy run (replay anchor)	it2i	equal	equal	passes	0

Restoring the injected conditioning (UniRL pull request #547) affected both rollout latency and initial judge scores. Generation time for 16 edits per rank decreased from 49.2 s to 36.8 s (25%), because the engine's fallback path encoded the source image into nearly four times as many vision tokens as UniRL's encoder path (5,938 versus 2,263 total context tokens). Simultaneously, the mean EditScore reward on the first rollout—evaluated on identical prompts prior to any optimizer update—increased from 2.80 to 4.62 (Figure 4), reflecting the quality difference between the two conditioning representations on the pretrained checkpoint.

7.2 Per-Group Output Canvas Resolution

Previously, every rollout batch required a single uniform output resolution fixed by the recipe, because the trajectory's latent segment stores latents as a single dense tensor and the rollout adapter passed a

single canvas size to the engine. Production prompt datasets, however, contain prompts with varying target aspect ratios and resolutions; forcing a single resolution either wastes compute on low-resolution prompts or truncates high-resolution ones.

UniRL pull request #549 allows each dataset example to specify its own target canvas dimensions in its metadata M_0. During rollout, the trajectory resolves the target canvas for each frontier row from its root ancestor (falling back to the recipe default), and the rollout adapter dispatches one engine call per contiguous run of rows sharing the same canvas size. To keep the latent segment tensor dense across a mixed-resolution batch, smaller canvases are right-padded along the token dimension to match the recipe's maximum canvas size, and the trainer slices each row back to its true token count during replay; canvases exceeding the recipe maximum are rejected at admission. Because all sibling samples forked from a root prompt share the same canvas, group-relative credit assignment, the micro-batched `RewardStack` of Section 6.3, and downstream scorers require no modification. Datasets without per-example canvas metadata execute identically to the fixed-resolution path. Rollout backends that support only a single resolution per batch—including the train-side diffusion adapter, non-BAGEL serving adapters, and video pipelines—explicitly reject the per-row canvas metadata field rather than silently rendering at the default resolution.

This padding scheme applies to objectives whose replay anchor stores a single scalar per denoising transition (such as Flow-GRPO). Objectives that store per-token replay anchors or consume the dense latent field directly still require a uniform canvas across the batch and now fail fast with a descriptive error when given mixed-canvas inputs.

Table 6: Generation time (seconds) for one prompt group of four sibling samples across three output canvas resolutions on a single H20 rank, for image-to-image editing (`it2i`, which additionally encodes a 512×512 source image) and text-to-image generation (`t2i`). The 768×768 `t2i` entry was measured on an earlier commit of the same branch.

Canvas	Image tokens	it2i, one per sample	it2i, packed	t2i, packed
512×512	1,024	9.05	7.25	6.50
768×768	2,304	17.45	15.76	14.55
$1{,}024 \times 1{,}024$	4,096	30.48	28.83	27.00

Table 6 shows that generation latency scales approximately linearly with the number of image tokens, creating a $4\times$ cost difference between 512×512 and $1{,}024 \times 1{,}024$ canvases within the same batch. When each data-parallel rank receives two prompt groups drawn in dataset order from a mixed-resolution prompt set, the slowest rank requires 1.16–1.84$\times$ the mean per-rank generation time, and the synchronous rollout barrier forces all other ranks to wait.

7.3 Sibling Request Packing for Image Editing

Group-relative credit assignment generates multiple sibling samples per prompt. In the original BAGEL image-editing adapter, each sibling sample was dispatched as an independent engine request, causing the engine to recompute the same vision-encoder passes and key–value conditioning contexts for every sibling in the group. With sibling request packing (UniRL pull request #550), when guidance scales permit, the adapter consolidates contiguous sibling rows from the same prompt group into a single engine request; the engine encodes the source conditioning once and broadcasts the resulting context across all siblings in the group. At 16 edits per rank (512×512), packing reduces per-rank generation time from 36.5 s to 28.4 s (22%, Table 7) at the cost of roughly 2 GiB of additional peak GPU memory, while a single group of eight siblings drops from 18.04 s to 13.96 s. The text-to-image path already packed sibling requests and is unaffected.

Because initial noise tensors are indexed per row, a packed request can span any contiguous subset of a sibling group rather than requiring the full group at once. Consequently, the adapter no longer requires `RewardStack` (Section 6.3) to round micro-batch sizes up to full group boundaries: a four-row micro-batch from an eight-sibling group executes as a single four-sample packed request.

7.4 Combined Effect of the Four Changes

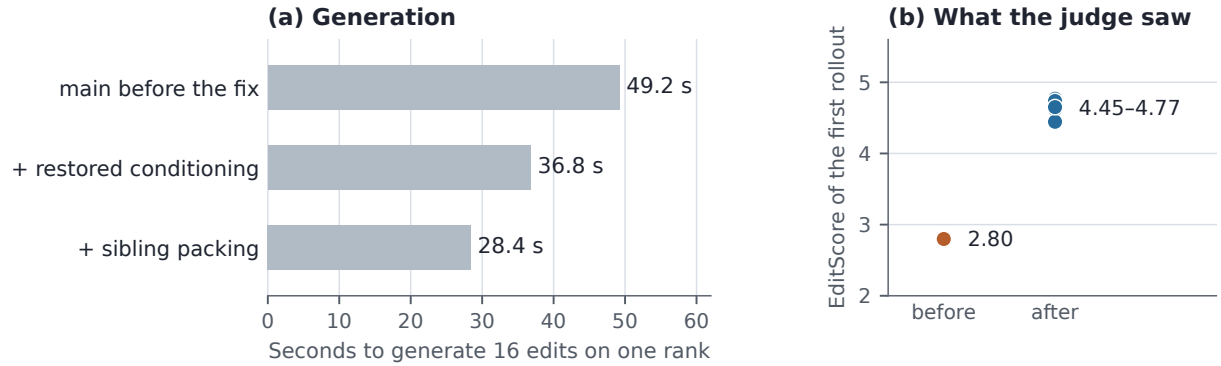


Figure 4: Cumulative effect of the four rollout-fidelity changes on BAGEL image editing (`it2i`) on one eight-GPU H20 node with a remote EditScore 8B judge. Left: per-rank generation time for 16 edits. Right: mean judge score on the first rollout prior to any optimizer step; points after the fix represent independent runs on the same initial prompt batch.

Table 7: Cumulative ablation of the four rollout-fidelity changes on BAGEL image-to-image editing (`it2i`) with LoRA on one eight-GPU H20 node (vLLM-Omni rollout, 16 prompts × 8 samples at 512×512, remote EditScore 8B judge). "Generation" reports the slowest rank's generation time for its 16 rows; "First-rollout reward" reports the mean judge score before the first optimizer update.

Rollout path	Rollouts	Generation (s)	vs. `main`	First-rollout reward
`main` before the fix	1	49.2	–	2.80
+ restored it2i conditioning (#547)	1	36.8	−25%	4.62
+ per-group canvas (#549)	1	36.5	−26%	4.77
+ sibling packing (#550)	4	28.4	−42%	4.45–4.74

Table 7 and Figure 4 summarize the cumulative impact of UniRL pull requests #547–#550 on the 512×512 image-editing workload: per-rank generation time drops by 42% (from 49.2 s to 28.4 s), while the first-rollout EditScore mean increases from 2.80 to 4.45–4.77. Across all arms, the first-update importance ratio remains within 10^{-4} of 1, and the structural token-layout check (enabled from #548 onward) passes on every replay, including configurations where `RewardStack` splits an eight-sibling group across micro-batches. As in Section 6.4, these measurements comprise one to four rollouts per arm on a single workload shape and establish the mechanism-level effect rather than multi-seed training variance.

7.5 Mitigating Mixed-Resolution Stragglers via Cost-Aware Assignment

When a rollout batch mixes output canvas resolutions, static data-parallel sharding—assigning prompts in dataset order with equal row counts per rank—creates severe load imbalance across ranks. On a mixed-resolution dataset whose canvases span a 4× range in image tokens, the default static split left at least one GPU rank idle for 29% of the text-to-image rollout window while waiting for the rank assigned the largest canvases.

To quantify how much of this straggler tail can be recovered without changing trajectory semantics, we evaluated a cost-aware work-assignment prototype. A driver-side coordinator orders prompt groups by estimated token cost and dispatches work units to ranks from a shared queue; when a rank finishes its assigned queue, work stealing allows it to claim the lowest-cost remaining unit from the busiest rank's

queue. Before credit assignment, completed rows are restored to their canonical parent-contiguous order by sample identifier I_i, leaving the trajectory structure consumed by the trainer unchanged. Table 8 reports steady-state rollout measurements across static, balanced, and work-stealing assignments.

Table 8: Comparison of rank assignment strategies on mixed-canvas batches (second rollout of each arm on one eight-GPU H20 node) for text-to-image (`t2i`) and image-to-image editing (`it2i`). "Idle" denotes the fraction of the rollout window during which at least one rank has completed generation and waits at the barrier. The cost model estimates work by output image tokens. Evaluated on an experimental branch without an open pull request.

Assignment	Rollout + reward (s)	Slowest rank (s)	Mean rank (s)	Idle
t2i, in-process PickScore				
Prompt-order shards (default)	150.2	147.2	103.8	29%
Cost-balanced shards	108.6	105.8	103.7	2%
Cost-balanced shards with stealing	108.7	105.8	103.6	2%
it2i, EditScore 8B on another node				
Prompt-order shards (default)	99.3	91.0	63.7	29%
Prompt-order shards, reward overlapped	93.7	91.2	63.7	30%
Cost-balanced shards with stealing	77.4	68.0	63.7	6%
Cost-balanced with stealing, reward overlapped	71.5	68.1	63.8	5%
it2i, EditScore 72B on another node				
Prompt-order shards (default)	128.4	90.8	63.7	21%
Cost-balanced with stealing, reward overlapped	131.1	68.4	63.8	8%

Three observations follow from two rollouts per arm:

1. **Static cost balancing removes most straggler idle time when the cost model is accurate.** In text-to-image (`t2i`) generation, cost-balanced assignment reduces rollout-plus-reward time from 150.2 s to 108.6 s and cuts the barrier idle fraction from 29% to 2%, while the mean per-rank generation time remains constant at approximately 104 s, confirming that work is balanced across ranks rather than reduced.
2. **Work stealing provides robustness against cost-model misspecification.** Under the image-token cost model, dynamic stealing was never triggered in `t2i` and transferred at most 2 units per rollout in `it2i`. However, under a deliberately misspecified cost model that counts only rows and ignores canvas resolution—which forfeits 25 s of the balancing reduction—work stealing recovers 18 s of that loss (reducing rollout time from 133.9 s to 115.9 s).
3. **Cost-aware assignment composes with generation–reward overlap only when the reward judge has spare capacity.** Against the 8B EditScore judge (which has headroom), reward overlap alone reduces `it2i` rollout-plus-reward time from 99.3 s to 93.7 s, cost-balanced assignment with stealing reduces it to 77.4 s, and combining both reduces it to 71.5 s (28% total reduction).

By contrast, when paired with the saturated single-replica 72B EditScore judge, cost-aware assignment *increases* combined rollout-plus-reward time from 128.4 s to 131.1 s with overlap (132.2 s without overlap) in the steady rollout, and from 118.3 s to 131.5 s (133.3 s without overlap) in the first rollout. Although balancing still reduces the slowest rank's generation time from 90.8 s to 68.4 s, synchronizing rank completion causes all ranks to submit their final reward requests simultaneously, concentrating the judge queueing delay that the imbalanced baseline had naturally staggered. This is the inverse of the request-staggering effect analyzed in Section 6.4, and implies that dynamic load balancing should be gated on available reward-service headroom (or paired with sufficient judge replicas and cross-request batching).

A second effect remains unaddressed in this prototype: in every arm that reordered rows across ranks, the subsequent training phase ran 1–5% slower in the steady rollout and 3–12% slower in the first rollout. Control runs that routed prompt-order shards through the same queue without cross-rank reassignment exhibited a 2–3% training slowdown, indicating that part of the overhead stems from the queue path itself and the

remainder from cross-worker latent transfers when rows are restored to canonical order before training. While total step time in the steady rollout remained shorter whenever balancing reduced rollout-plus-reward time, in the first rollout the training slowdown outweighed the rollout gain under the 8B judge without overlap (increasing total step time by 2.7 s). Preserving the balanced rank assignment into the training phase would avoid cross-worker reordering and remains future work.

8 Evaluation: Reported Results and Remaining Evidence

We organize the main evaluation around four research questions (Table 9). Throughout the synchronous training and systems benchmarks below, one outer-loop iteration collects one rollout batch and applies the configured optimizer updates; we therefore refer to training progress in *rollouts* (and to per-rollout wall-clock duration in systems benchmarks as *iteration* or *step* time). Each reported run archives the resolved configuration, source commit and patch state, model and dataset revisions, environment manifest, structured metrics, and evaluation outputs (Appendix C). Primary comparisons use the same hardware one framework at a time.[1]

Evidence scope and date. The initial GPU experiment bundle, archived on 2026-09-27 with an evidence cutoff of 2026-09-22, provides source snapshots, transcribed summaries, and deterministic figure/table generators under `artifacts/reported_results/2026-09-22/`. The E2 and diffusion E3 tables below are regenerated from those reported summaries rather than raw per-step training logs. The later E3-AR study was rerun and is archived separately under `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/`, including resolved recipes, the recipe and effective-work audit, run identities, post-warm-up per-rollout records, phase and band summaries, and the deterministic plotting script. The HunyuanImage-3.0 and MiniMax-H3 recipe reproductions in Section 8.1 are likewise generated directly from their exported W&B histories and resolved training recipes. We maintain the distinction between report-derived and directly exported evidence and avoid interpolated learning curves or unsupported uncertainty bars. Unless explicitly identified as an observed measurement, protocol descriptions below state the prespecified experimental design.

Table 9: Experiment-to-claim map and evidence status through 2026-10-11. P0 rows remain necessary for the primary trajectory-centric claim.

ID	Priority	Controlled study	Paper evidence
E0	P0 gate	Correctness, configuration, and instrumentation checks	Correctness closure reported; evidence packaging remains incomplete
E1	P0	Qwen3-4B-Base on DAPO-Math	GRPO and DRPO each trained at one seed; primary external endpoint missing
E2 / E2-R	P0	SD3.5-Medium diffusion training (UniRL E2 and VeRL-Omni reference E2-R)	Three-seed UniRL scores and diversity reported (E2); VeRL-Omni reference trained, not evaluated (E2-R)
E3	P0	Native UniRL–VeRL-Omni recipes, eight GPUs	Aggregate timings reported; strict semantic parity not established
E3-AR	P0	Recipe-matched Qwen3-4B-Base GRPO, UniRL–veRL, 16 GPUs per system	500 rollouts complete; resolved recipes, parity audit, per-rollout measurements, and curves released
E4 (E4a/E4b)	P0	Root versus rewrite grouping: synthetic oracle check (E4a) and 3-seed training (E4b)	Synthetic equality checks pass (E4a); no consistent three-seed quality advantage (E4b)
E5 (E5-R/E5-T)	P0 minimum	Paired IR representation cost (E5-R) and colocated vs. separate deployment topology (E5-T)	Deployment topology measured (E5-T); paired representation cost not measured (E5-R)
E6 (S0/D0/A1)	P1	Qwen3-4B-Base colocated sync (S0), separate sync (D0), and async overlap (A1)	Single-seed pilots only; formal sweep canceled

The execution priority is E0, followed by E1–E4 and the minimum E5 topology/cost slice (E5-T and E5-R); E6 and broader E5 scaling/transport backends follow. When compute is constrained, completing primary E0–E5 takes precedence over adding models, algorithms, or scaling points: without E4 the evaluation would not directly test cross-stage lineage, and without E5-R and E5-T it would not bound the cost of that abstraction.

[1]Throughout the paper, we follow each baseline project's official capitalization: *veRL* for the language-model post-training framework (Sheng et al., 2024) and *VeRL-Omni* for the multimodal and diffusion extension (Huang et al., 2026).

Environment and source identity. E3 and the deployment-topology comparison E5-T use a fixed allocation of eight GPUs; complete hardware, interconnect, environment, model, data, and scorer manifests remain to be attached to those legacy summaries. E3-AR uses two eight-GPU H20 nodes per framework and releases its pinned source commits and resolved recipes alongside the per-rollout records. Hardware specifications from the 32-GPU E1 allocation are not extrapolated to E2 or E3, and the v0.2.0 design-audit commit is kept distinct from the commits used by later experimental runs.

8.1 RQ1: Does UniRL learn on representative workloads?

The first question is scientific rather than purely operational. A complete training-validity claim requires improvement over the frozen base together with satisfactory collapse diagnostics. E2 reports an improvement in automatic scores but triggers the diversity guard, so its result is a qualified quality–diversity trade-off. Reproducing another implementation additionally requires a full-budget, mathematically aligned reference with common quality endpoints; a systems speed run alone cannot supply that evidence. The VeRL-Omni reference training run (E2-R) has been trained across three seeds but has not been evaluated head-to-head on held-out prompts, while the adapted AR systems launcher (E3-AR) has not yet been evaluated on the aligned external quality benchmark.

AR workload. E1 trains Qwen3-4B-Base (Yang et al., 2025) with full-parameter GRPO on DAPO-Math-17k (Yu et al., 2025). The dataset name does not imply use of the DAPO optimizer; we fix the population-standard-deviation group estimator implemented in the audited recipe. Each rollout contains 64 prompts with eight samples per prompt (512 trajectories), uses SGLang TP=1 with thinking enabled, temperature and top-p 1.0, top-k disabled, and at most 8,192 new tokens. The learner uses group-standard-deviation normalization, symmetric clipping at 0.2, per-sequence token-mean loss (`seq-mean-token-mean`), four optimizer updates per rollout under a 10,240-token per-GPU micro-batch budget, AdamW learning rate 10^{-6} and weight decay 0.01, and full-weight publication every rollout. We prespecify 800 rollouts on 32 total GPUs, internal AIME evaluation every ten rollouts, full checkpoints every 200 rollouts, and seed IDs 11, 22, and 33 applied to data order and the SGLang server random number generator (RNG). These seeds and the checkpoint cadence are supplied as run-time overrides because the stock recipe uses data-source seed 42; at the audited release a per-request SGLang sampling seed requires deterministic inference, so the server RNG is seeded through engine arguments recorded in the resolved configuration. The prespecified external evaluation covers the frozen base and checkpoints at rollouts 200, 400, 600, and 800 on MATH-500 (500 problems, avg@4) and AIME 2024/2025 (30 problems each, avg@16), using fixed decoding parameters and the `math_verify` grader. The UniRL run and the reference baseline must align prompts, chat template, verifier, maximum generation length, group size, effective batch, clipping, KL treatment, optimizer updates, and evaluation decoding. The public veRL Qwen3 FSDP example serves only as a starting scaffold because its stock model variant, dataset, group size, response budget, and KL/objective settings differ. E3-AR uses a pinned adapted launcher whose resolved recipes are archived and checked field by field against fifteen cross-framework effective-work identities in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/recipe_parity_audit.json`; two residual estimator-level differences that configuration cannot remove are quantified there, and the bundle documentation lists the recipe items outside those fifteen identities. Absent frozen-base and checkpoint evaluations under the common external protocol, E3-AR does not by itself establish quality reproduction. The final 800-rollout checkpoint is primary. Internal evaluation uses AIME 2024/2025, so an AIME-selected best checkpoint is reported only as validation-selected and cannot establish an untouched AIME test result. MATH-500 is the primary external AR endpoint, with a training/evaluation overlap audit required before interpretation; the fixed-budget AIME curves are secondary diagnostics. The protocol calls for every seed, final-checkpoint curves, and 95% confidence intervals; avg@k denotes mean accuracy across k sampled answers, not pass@k.

E1 status: training complete, primary endpoint open. The current evidence records two full-budget single-seed arms: standard GRPO and an additional Divergence-Regularized Policy Optimization (DRPO) arm, which replaces asymmetric ratio clipping with a smooth advantage-weighted quadratic trust-region penalty. Internal AIME accuracy changes from 0.0208 to 0.0177 for GRPO and from 0.0250 to 0.159 for DRPO over the 800-rollout run (following an earlier verifier-process cleanup fix in UniRL pull request #462). Both arms improve in mid-training and subsequently decline; these internal validation measurements use a

smaller decoding budget than the primary MATH-500 endpoint. Because the external MATH-500 evaluation and multi-seed replication remain pending, these runs do not establish objective superiority or general AR training validity.

Diffusion workload. E2 fixes SD3.5-Medium (Stability AI, 2024) and the UniRL Flow-GRPO setting used by E3, i.e. the stock `sd3_vllmomni.yaml` recipe with the alignment overrides in `run_unirl_sd35_aligned.sh`: 48 prompts with 16 images (768 samples) per rollout, 384×384 training resolution, ten denoising steps, three SDE steps in the first half, $\eta = 0.8$, guidance 1, and distinct initial noise. PickScore (Kirstain et al., 2023) supplies the training reward; advantages subtract each prompt-group mean and divide by the batch-wide reward standard deviation. LoRA rank/alpha is 32/64 on eight matched attention projections; the learner uses two updates, micro-batch size eight, learning rate and weight decay 10^{-4}, clip 10^{-5}, and weight publication every rollout. We prespecify 300 rollouts on one eight-GPU node with seed IDs 11, 22, and 33. The same budget and seed IDs apply to the pinned VeRL-Omni reference run (E2-R); checkpoint export and estimator parity are audited before admission. Because the stock configurations differ in SDE transition strategy—UniRL uses `FlowSDEStrategy` whereas VeRL-Omni uses Coefficients-Preserving Sampling (CPS)—as well as step selection and normalization/anchor semantics, E2 and E2-R currently define a related-recipe comparison. At least one full-budget E1 or E2 reference must pass mathematical and work parity before the paper claims reference-implementation reproduction. Frozen base and every 50-rollout checkpoint use a shared diffusers evaluation pipeline. The primary image endpoint is PartiPrompts HPSv3 (Ma et al., 2025b); ImageReward (Xu et al., 2023) and compositional scores provide complementary checks. These are held-fixed automatic judges distinct from the optimized PickScore model, not independent human studies.

Image evaluation identity. The registry key `image/geneval2` points to UniRL's `synthetic/test.jsonl` and a Qwen3-VL-8B Soft-TIFA-style scorer, served by the reward service by default or run in-process with `-local-geneval2`. We name this the *synthetic compositional set*; it is not an official GenEval 2 result (Kamath et al., 2025) without a separate data-and-scorer parity audit. The prespecified image endpoints use 512×512, 40 steps, guidance 1, and an evaluation seed of 42 shared across training seeds and checkpoints. The E2 evaluation summary uses 16 images for each of 1,632 preference prompts, rather than a single-image preference endpoint; the exact sampler and aggregation revision are recorded in the artifact bundle. The synthetic endpoint additionally fixes its explicit linear sigma grid, prompt-hashed seeds, and 256-token text-encoder limit; the preference endpoint fixes the pinned pipeline schedule and index-based seeds. Dataset sizes, revisions, overlap counts, and scorer hashes remain open manifest fields.

The diversity guard is within-prompt mean pairwise LPIPS (Zhang et al., 2018) (AlexNet features), computed over 16 images per PartiPrompts prompt at the same evaluation setting, with identical image seeds for base and checkpoint. We average all 120 pair distances per prompt, then report the relative change in the mean across prompts and a paired prompt-bootstrap interval. A greater than 10% relative decrease is a study-specific investigation trigger; it is not a literature-established quality threshold. A near-zero base makes this relative criterion undefined and requires the absolute change to be reported. This guard diagnoses reduced diversity and is not itself a perceptual-quality metric.

Table 10: Reported E2 diffusion endpoints after 300 rollouts, with three training seeds. Changes are absolute score differences unless marked as relative percentages. Missing base/final levels are not reconstructed from differences.

Metric	Base	Rollout 300	Δ score	Δ relative
HPSv3	1.2024	9.8126	+8.6102	–
ImageReward	–	–	+1.3932	–
PickScore	–	–	+0.0970	–
Composition (VQAScore)	0.1011	0.1552	–	+53.5%
LPIPS diversity	–	–	–	-24.79%

E2 result: higher automatic scores with lower diversity. The reported mean HPSv3 gain is 8.6102, while within-prompt LPIPS changes by -24.79%. Figure 5 shows the available individual-seed differences, keeping score improvement and diversity reduction adjacent. The report records complete scoring with no errors; the preference evaluation contains 1,632 prompts and 16 images per prompt, not that many independent training runs. Synthetic composition and preference use different evaluation sets, so the preference image count must not be assigned to every metric. All three seeds exceed the prespecified diversity-investigation threshold. The reported reduction is already present at rollout 50; stopping at that checkpoint does not eliminate it, although these observations do not rule out earlier stopping or a different intervention. External automatic judges also improve, but this does not establish human preference or exclude reward exploitation. The supported conclusion is a score–diversity trade-off, not an unqualified training-validity or no-collapse result. Only endpoint summaries are available in this bundle; no intermediate learning curve is interpolated.

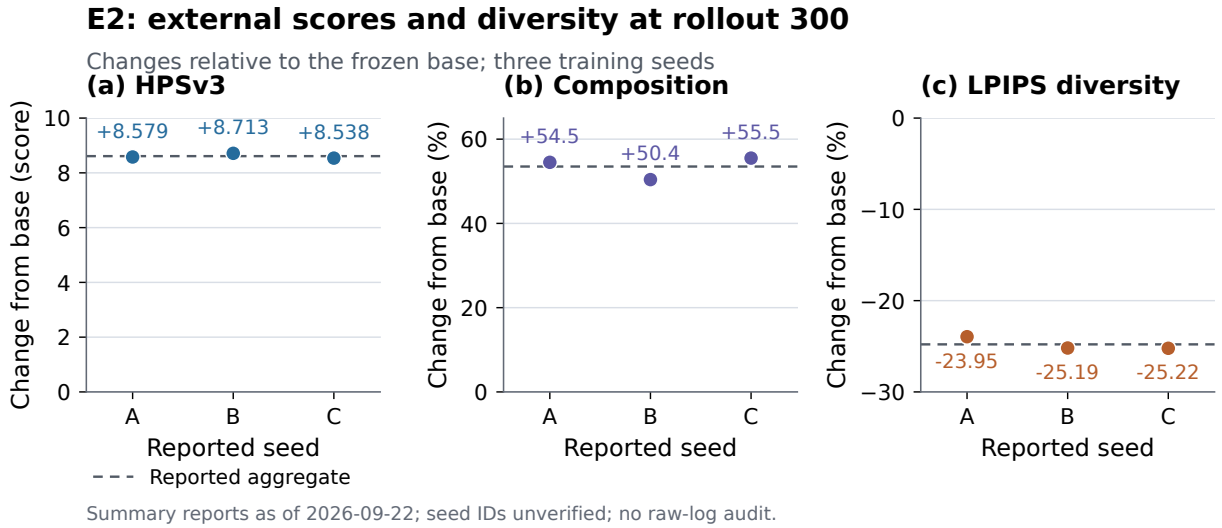


Figure 5: Reported per-seed E2 changes at the final checkpoint relative to the frozen base. Each panel retains its own metric and units; relative composition changes are not percentage-point changes. Points are training-seed summaries, not individual images. These are report-derived endpoints, with no raw-data confidence bands.

Reference comparison (E2-R) remains open. The VeRL-Omni reference run E2-R completed three training seeds and verified checkpoint exports, but no held-out head-to-head evaluation was performed. Export correctness cannot establish quality parity, and evaluation alone would not remove the native flow versus CPS estimator mismatch. An explicitly labeled native-recipe comparison could provide context; strict reproduction still requires mathematical and effective-work parity.

HunyuanImage-3.0 recipe reproduction. We further evaluate UniRL on HunyuanImage-3.0 (using the `HunyuanImage-3.0-Instruct` checkpoint) as a high-resolution text-to-image workload. The run is single-stage diffusion: the train-side recipe samples and replays only the image-generation mode, with no AR rollout or AR objective, so it does not exercise the shared-backbone AR and diffusion trajectory described in Sections 3.4 and 3.5. It uses one node with eight H20 GPUs, 16 prompts and four samples per prompt, producing 64 images per rollout at $1{,}024 \times 1{,}024$ resolution. Sampling uses 16 inference steps and three SDE steps, and training uses Flow-GRPO with PickScore reward and seed 42. The exact resolved recipe is archived under `artifacts/reported_results/2026-09-30-hunyuanimage3/`.

We report the run through its durable checkpoint at rollout 360; the archived history continues to rollout 369. Mean training reward increased from **0.762** over the first ten rollouts to **0.870** over rollouts 351 through 360 (Figure 6). This result shows that the HunyuanImage-3.0 text-to-image recipe executes and improves

its training reward in UniRL. Because the run uses one seed and has no held-out image-quality evaluation, we treat it as recipe-reproduction evidence rather than a comparative quality result.

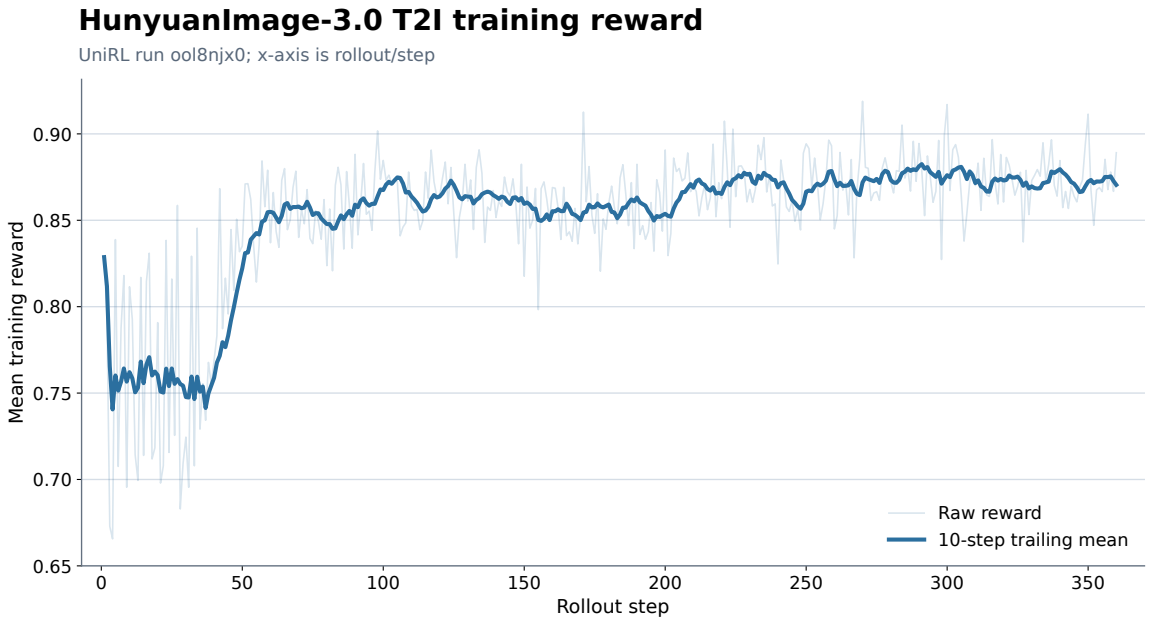


Figure 6: HunyuanImage-3.0 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL's rollout counter (`rollout/step`), rather than W&B's default `_step`.

MiniMax-H3 recipe reproduction. We also evaluate UniRL on MiniMax-H3 text-to-video-with-audio generation. Each rollout contains eight prompts with 16 samples per prompt, or 128 trajectories, generated at 256×384 resolution with 124 frames and ten denoising transitions. Training uses one optimizer update per rollout, learning rate 10^{-4}, ImageBind and CLAP rewards, seed 42, and one node with eight H20 GPUs. The ImageBind term scores audio–video correspondence on the middle frame and the CLAP term scores text–audio agreement; neither scores prompt–video alignment or temporal video quality. The resolved configuration is archived under `artifacts/reported_results/2026-09-30-minimax-h3/`.

We report the run through its durable checkpoint at rollout 200; the archived history continues to rollout 204. Mean reward increases from **0.194** over the first ten rollouts to **0.375** over rollouts 191 through 200 (Figure 7). Because this is a single-seed run with no held-out video and audio quality evaluation, we use it as evidence of executable recipe and model coverage rather than as a comparative-quality, systems-performance, or time-to-quality result.

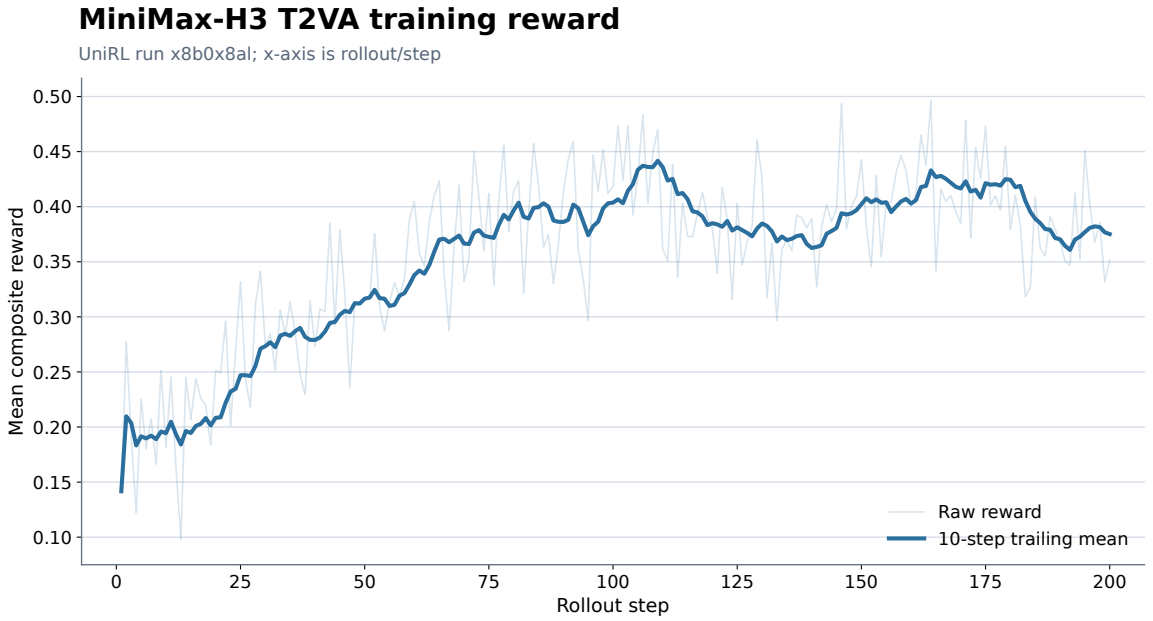


Figure 7: MiniMax-H3 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL's rollout counter (`rollout/step`), rather than W&B's default `_step`.

8.2 RQ2: What is end-to-end system performance?

We report two complementary performance comparisons. E3 compares UniRL with VeRL-Omni commit `01c87ee5` for the E2 SD3.5-Medium shape-matched work unit on one reserved eight-GPU node (Huang et al., 2026); E3-AR is a separately rerun, recipe-matched Qwen3-4B-Base GRPO comparison against veRL. The planned E3 work unit uses 48 prompts, 16 samples, 384×384 output, ten denoising steps, three early-window SDE steps, two optimizer updates, micro-batch size eight, LoRA 32/64, learning rate/weight decay 10^{-4}, clip 10^{-5}, and PickScore. Because the two frameworks target different attention projection sets in their stock LoRA configurations, matching rank and alpha does not establish identical trainable parameter counts. The reported protocol uses three process-level repetitions per configuration, 30 rollouts each, removes the first five timing observations as warm-up, and retains at least 20 observations per repetition. Systems run separately on the same reserved node; cache policy and randomized execution order remain checks against the underlying run records. Table 11 reports the three distinct configurations: UniRL with its native flow strategy and scaled dot-product attention (SDPA), and VeRL-Omni with its native CPS strategy under both SDPA and FlashAttention-3 (FA3). The stock UniRL recipe uses `FlowSDEStrategy` and three sampled early indices; the VeRL-Omni launcher requests `cps` and an early three-step window. These differences change the transition distribution and likelihood, not merely the kernel implementation. Normalization, anchors, actual replay work, and reward-service GPU placement also require an audit. At the audited release the stock SD3 recipes also enable update-membership shuffling (`shuffle_updates`), which the reported runs predate; a rerun at v0.2.0 must either disable it or report it as a further difference. Until a semantically aligned pair is established, E3 is labeled a *shape-matched native-recipe comparison*; attention-backend matching alone does not make it an equivalent-algorithm framework comparison. Its speed ratio cannot be attributed solely to framework overhead.

A recipe-matched AR systems comparison (E3-AR) is reported below; the primary MATH-500 quality endpoint and a controlled time-to-quality comparison remain open.

The full protocol requests median, mean, and p90 iteration time, throughput, phase timing, memory, and utilization. The older report-derived E3 bundle supplies only selected aggregate timings and throughput,

whereas E3-AR supplies the post-warm-up per-rollout timing and response-length distribution across 499 measured rollouts (one process-level run per framework). Strong and weak scaling use fixed global and fixed per-GPU work, respectively, after the primary single-node pair is complete. The 30-rollout E3 protocol does not support a time-to-quality claim; the E2-R quality comparison is still missing and E6 did not complete a threshold-based study.

Table 11: Reported E3 native-recipe systems timings and E5-T deployment-topology timings. All rows use eight allocated GPUs. E3 and E5-T answer different questions; differences in estimators and deployment prevent a pure framework or transport attribution. Aggregate values retain the source table's precision, not an independently established measurement accuracy.

System	Configuration	s/iteration	samples/GPU-h
E3: native recipes, eight GPUs in total			
UniRL	Native flow / SDPA-class	64.2	5381
VeRL-Omni	Native CPS / SDPA	111.3	3104
VeRL-Omni	Native CPS / FA3	112.3	3079
E5-T: deployment regimes, eight GPUs in total			
UniRL	Colocated, 8 shared GPUs	64.223	–
UniRL	Separate, 4 training + 4 rollout GPUs	146.244	–

E3 result and its comparison boundary. The reported UniRL and VeRL-Omni SDPA times are 64.22 and 111.33 seconds per iteration, a ratio of approximately 1.73. The observed SDPA and FA3 times are similar; this is not an equivalence test. The result concerns the complete measured native configurations, including their estimator and LoRA-target differences. The report also records greater short-run reward improvement per iteration for VeRL-Omni, so iteration throughput cannot be converted into a time-to-quality advantage. Neither a parameter-adjusted ratio nor a progress-normalized ratio is presented as an independently validated matched-control result. Generation phase labels are also non-equivalent: VeRL-Omni includes scoring in its generation timer, whereas UniRL separates it. We therefore omit a phase-breakdown figure until a common accounting boundary and the underlying measurements are available. Because the legacy report summaries contain minor discrepancies across replicate notes, Figure 8 plots only the aggregate table values without inferred replicate points, confidence intervals, or statistical significance claims.

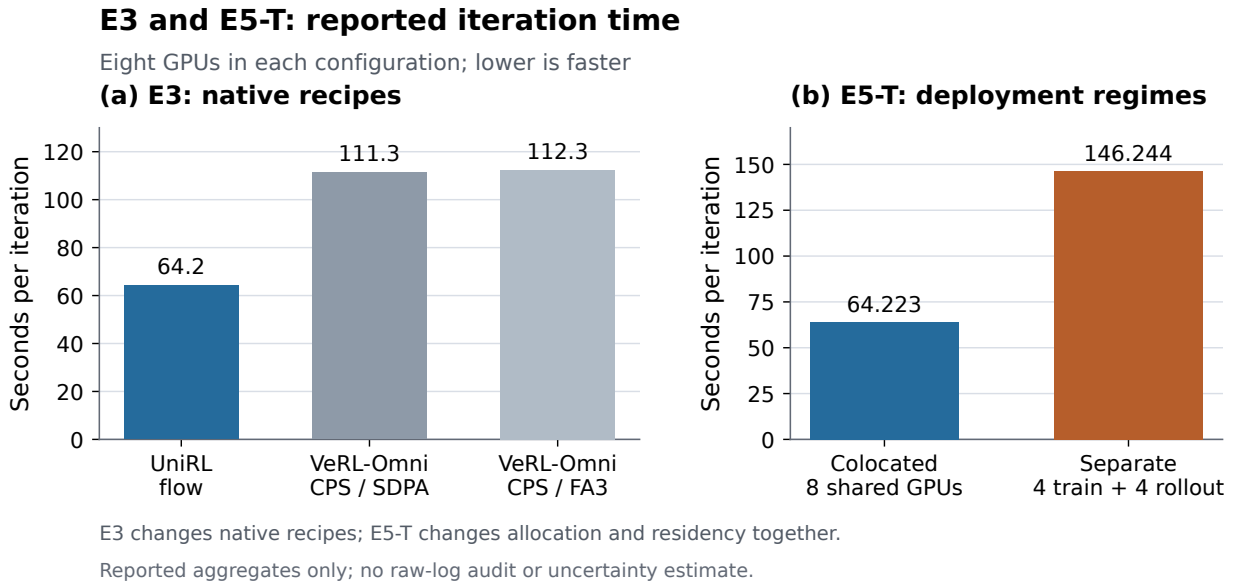


Figure 8: Reported iteration-time summaries. Left: E3 native-recipe comparison on one eight-GPU node. Right: E5-T's separate four-training/four-rollout allocation versus eight colocated GPUs within UniRL. Lower is faster. Panels keep the external recipe comparison separate from the internal deployment comparison; no error bars are inferred from unavailable run-level data.

Recipe-matched autoregressive comparison (E3-AR). We additionally compare UniRL and veRL on a 500-rollout Qwen3-4B-Base (Yang et al., 2025) standard GRPO workload over DAPO-Math-17k (Yu et al., 2025), matching the prespecified RQ1 algorithm. Each framework uses 16 NVIDIA H20 GPUs across two eight-GPU nodes. The matched recipe uses 64 prompts with eight samples per prompt, temperature and top-p 1.0, top-k disabled, natural EOS termination with an 8,192-token response cap, and four optimizer updates per rollout under a 10,240-token per-GPU micro-batch budget. The two runs align the model, dataset and seeded prompt shuffle, tokenizer, verifier, GRPO objective (advantage divided by the group standard deviation, per-sequence token-mean loss (`seq-mean-token-mean`), symmetric clip $\epsilon = 0.2$), rollout log probabilities as the frozen π_{old} anchor, optimizer settings, fp32 master weights with bf16 compute, rollout batch, and update count. UniRL uses eight SGLang TP=2 engines, while veRL uses eight asynchronous vLLM TP=2 engines; these serving stacks are part of the compared end-to-end systems. The recipes do not align the training attention kernel (UniRL uses FlexAttention), reward placement (UniRL scores responses in a separate reward stage, whereas veRL's reward manager runs inside rollout), or realized response lengths, which follow each policy's own training trajectory. We exclude startup rollout 1 as warm-up and report rollouts 2–500 (499 measured iterations).

Table 12: Recipe-matched Qwen3-4B-Base GRPO performance over rollouts 2–500 (one rollout per iteration step). Recipes are matched; realized response lengths are not. Times and response length are mean ± step-level population standard deviation over 499 rollouts. Token rates divide total generated tokens by total measured time. Step-level deviations describe these runs and are not process-level uncertainty estimates.

System	E2E (s/step)	Generation (s/step)	Training (s/step)	Response tokens	E2E tok/s	Generation tok/s
UniRL	143.519 ± 28.601	**76.059 ± 15.129**	61.196 ± 13.458	4,197.6 ± 1,005.4	**14,975**	**28,257**
veRL	**139.894 ± 30.731**	80.619 ± 16.308	**55.743 ± 14.577**	3,781.8 ± 1,039.1	13,841	24,018

AR systems result. UniRL's mean raw end-to-end iteration time is 2.6% higher than veRL's while it samples responses that are 11.0% longer on average. The median raw end-to-end iteration time is 147.7 s for UniRL and 150.1 s for veRL, and the 90th percentiles are 172.6 s and 169.6 s. After accounting for this varying

token work, UniRL provides 8.2% higher end-to-end output-token throughput and 17.7% higher generation output-token throughput. The native veRL actor-update timer is lower than the UniRL training timer, whereas UniRL's generation advantage is larger for this workload. The raw end-to-end ordering reverses near rollout 300 as the two sampled response-length distributions cross, but UniRL's token-normalized end-to-end and generation throughput are higher in every one of the five 100-rollout bands. We therefore treat token-normalized end-to-end throughput as the primary result rather than claiming a latency advantage at every training point. Phase values use each framework's native instrumentation and are diagnostic; end-to-end time has the clearest common boundary. Two estimator-level differences remain that configuration cannot remove: UniRL divides advantages by the population standard deviation and veRL by the sample standard deviation, making veRL's advantages uniformly $\sqrt{7/8} = 0.935\times$ smaller at eight samples per prompt, and the two advantage epsilons differ by 10^{-8} versus 10^{-6}; both are quantified against their closed forms in the bundle's audit. This experiment contains one process-level run per system and does not establish repeated-run uncertainty or time-to-quality.

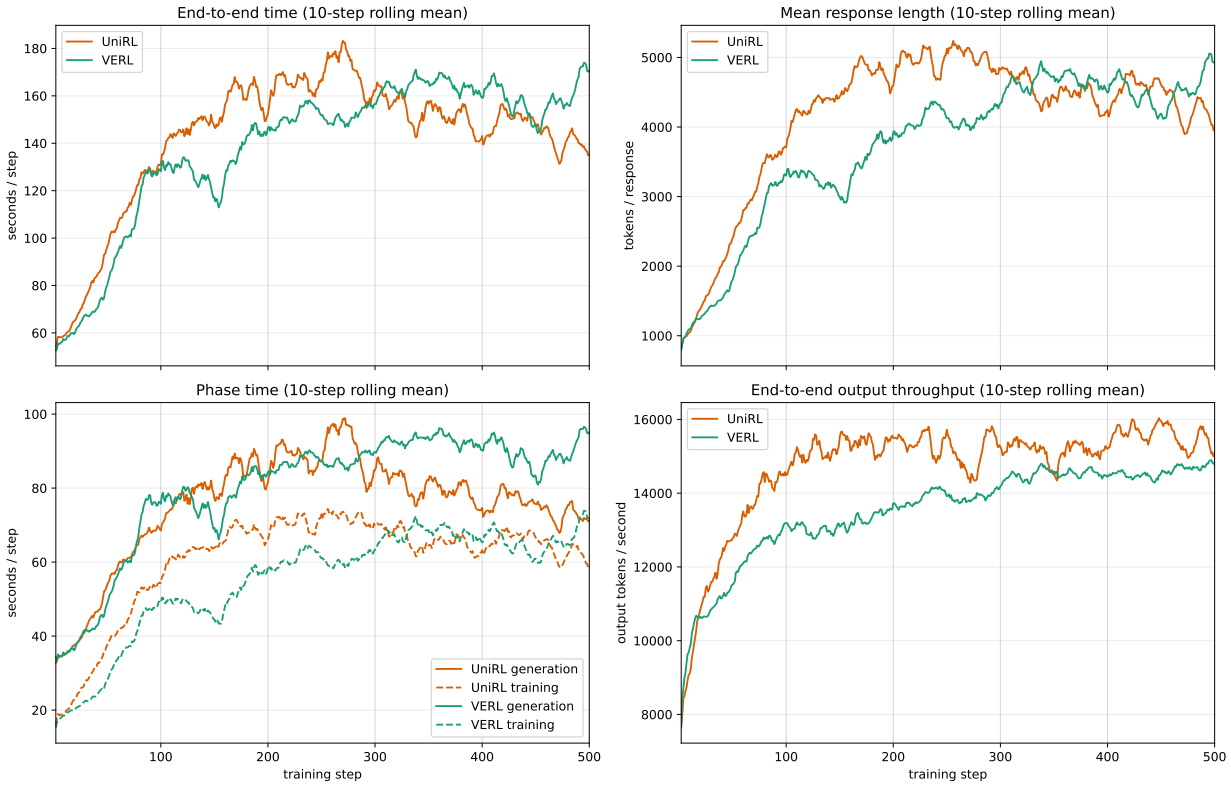


Figure 9: Ten-rollout rolling means for the 500-rollout natural-length Qwen3-4B-Base GRPO comparison. Rollout 1 is excluded as startup warm-up. Response lengths evolve during training and cross near rollout 300, reversing the raw step-time ordering; token-normalized end-to-end throughput remains higher for UniRL throughout the five reported bands.

8.3 RQ3: What does the trajectory abstraction cost and enable?

RQ3 first tests the semantic capability that motivates the IR (E4, comprising synthetic equality verification E4a and three-seed training comparison E4b), and then measures its representation and topology costs (E5, comprising the paired representation microbenchmark E5-R and the deployment-topology comparison E5-T). E4 uses the executable frozen-Qwen3-0.6B rewrite $\rightarrow$ SD3.5-Medium path: eight original prompts, four rewrites per prompt, and eight images per rewrite, or 256 image descendants per rollout; only diffusion trains. This is a separate recipe from E2: 512×512, ten steps, one sampled SDE index among 0–8, $\eta = 0.7$, LoRA 16/32, learning rate 3×10^{-4}, weight decay zero, clip 10^{-4}, replay anchor, two updates, and micro-batch size

one. The primary condition (`diffusion_group_scope=prompt`) computes group-relative advantages across all 32 descendants of one original prompt, while `diffusion_group_scope=rewrite` (the trainer default) groups the eight images under each rewrite. The two stock prompt-enhancement recipes are not a matched pair: the prompt-group recipe uses eight images per rewrite and a 512-token rewrite limit, whereas the rewrite-scope recipe uses four and 256. Both arms must therefore use the prompt-group recipe with only the grouping key changed, as confirmed by their resolved configurations; both stock recipes also enable `shuffle_updates` at the audited release. Both conditions must serialize redacted root/layer/parent IDs, shapes, reward/advantage, replay indices, and group labels; the synchronous prompt-enhancement trainer does not advance behavior versions, so they carry no behavior-version provenance here. Invalid lineage remains an E0 fail-fast test rather than an intentionally corrupted quality baseline.

For image $x_{p,j,k}$, the two group keys are $g_{\text{root}} = p$ and $g_{\text{rewrite}} = (p, j)$. The intended semantic test (E4a) requires an ID-joined flat-row reference to reproduce UniRL's group memberships, advantages, and learner inputs on identical recorded rewards and replay tensors, including after root-shard round trips. The 300-rollout, three-seed training comparison (E4b) changes the normalization population (32 versus eight images); any quality difference would therefore concern the grouping objective, not isolate the IR representation's effect.

Passing that recorded-trajectory control would establish the grouping semantics on real rollout data; it would not by itself prove better task quality. The current PickScore request is conditioned by the rewritten text (the default `prompt_source: generation`; scoring against the root prompt with `prompt_source: original` is available but was not used in E4), so a claim about preserving the user's original intent additionally requires lineage-keyed evaluation of every image against its root prompt. We therefore report both rewrite-conditioned training reward and frozen HPSv3/ImageReward evaluations joined to each root prompt. Evaluation generates held-out images at 512×512, 40 steps, and guidance 1 with the base and every 50-rollout diffusion checkpoint, using one fixed manifest of four frozen rewrites per root and eight image seeds per rewrite for both conditions. Worker/adapter RNG initialization, deterministic train-side AR sampling, real trace dumping, and this held-out generation/evaluation path are E0 prerequisites for a multi-seed E4 result. If the two correct groupings tie, the experiment still establishes semantic expressiveness but not a quality benefit.

E4 reported results and unresolved input validity. In E4a, all 24/24 synthetic checks pass in each grouping scope with bit-exact agreement against an explicit ID join. Because the fixtures are synthetic, E4a provides bounded semantic verification rather than the recorded-rollout check specified above. In E4b, the three-seed training comparison shows no consistent quality advantage between prompt-scope and rewrite-scope grouping: the score differences change sign across seeds. This outcome neither demonstrates equivalence nor establishes that the true effect is zero. Inspection of the E4b runs also identified malformed rewrite conditioning: chat template markers remained in many rewritten prompts and many rewrites exceeded the 77-token CLIP window because the intended text-extraction filter was not applied. Those defects can affect both absolute image quality and the grouping contrast; a statistically inconclusive outcome does not make them benign. We retain E4b as an exploratory observation until a corrected rewrite pipeline passes a pre-flight validation check.

Paired representation cost (E5-R) and deployment topology (E5-T). The intended E5-R microbenchmarks separate recursive IR operations and tensor transport from model compute, varying root count, branch factor, tensor payload, token/media lengths, and layer depth. Required outputs include structural-operation time, metadata bytes, dense bytes avoided at the driver, transfer time, and reassembly correctness hashes.

The broader E5 protocol compares the colocated store, per-GPU store, and TransferQueue path where hardware permits. The current harness does not yet implement the paired E5-R representation measurement: whole-tree sharding versus explicit ID-joined flat rows on the same recorded payload, device, transport, and operations. This paired representation slice holds model work, device placement, values, grouping, and replay inputs fixed; driver resident set size (RSS), bytes moved, operation time, and fraction of iteration cost

measure the price of preserving structure. Semantic errors are caught in E0 and are never timed as valid work.

The minimum topology slice (E5-T) reuses E3's colocated SD3.5-Medium UniRL row and compares it against the same workload under `sd3_vllmomni_lora_separate.yaml`, where four GPUs train and four GPUs run rollout within the same eight-GPU allocation. Because that stock separate-slab recipe differs from E3's aligned settings, the E3 overrides must be applied and archived with the separate arm. E5-T compares two complete deployment regimes rather than an isolated transport primitive: training DP width, memory residency, and local versus cross-slab LoRA publication change together and require a common phase breakdown before attributing their costs. Broader same-/cross-GPU and cross-node backend measurements remain E5 extensions. The reported E5-T iteration times are 64.223 seconds for colocated deployment and 146.244 seconds for the separate allocation, approximately 2.277 times longer (Figure 8). The separate UniRL configuration is also slower than the colocated VeRL-Omni E3 configuration. The experiment report attributes much of the increase to reward processing, but without exported phase records we retain this as a reported diagnosis rather than an isolated causal attribution. Neither this deployment penalty nor the CPU timings below establish bounded representation overhead. To document the $N_{\text{mod}} + N_{\text{alg}}$ composition in Section 3.6, the E4 path adds only workflow-specific bundle and trainer wiring while reusing the core trajectory IR, scheduler, transport, and credit modules unchanged.

Preliminary executable evidence. We ran a source-importing regression harness against a clean UniRL checkout at `fd344ad`; it does not mock the trajectory classes. All ten checks passed: cross-stage lineage construction, exact tree split/concat, whole-tree select/slice, leaf-to-root reward propagation and group advantages, rejection of malformed lineage and primitive modality/batch violations, variable-length packed-text round trips, and lazy `TensorRef` selection/materialization including bounds checking. Table 13 reports a deliberately narrow timing slice from the same run. The synthetic workload has four AR children per root, one observation per AR child, two diffusion children per observation, and a four-value image payload per leaf.

Table 13: Preliminary single-process CPU trajectory-IR timings (microseconds). Python 3.12.13 and PyTorch 2.11.0; one thread; ten warm-up and 100 measured repetitions. These are regression measurements, not distributed or GPU throughput.

Roots	Leaf rows	Operation	Median	p90
8	64	`sample.split_concat`	120.8	142.6
8	64	`sample.select_half`	556.8	616.7
8	64	`tensorref.select_strided`	12.3	13.0
32	256	`sample.split_concat`	291.2	329.6
32	256	`sample.select_half`	2231.5	3126.8
32	256	`tensorref.select_strided`	47.0	52.4
128	1024	`sample.split_concat`	1435.5	2231.7
128	1024	`sample.select_half`	8152.4	8458.7
128	1024	`tensorref.select_strided`	202.2	226.4

The current `Sample.select` path forms tree-complete root shards before gathering them, so its cost grows with the full input tree rather than only the selected rows. Strided `TensorRef` selection creates many metadata spans while contiguous selection coalesces them; no dense tensor is copied until materialization. These observations motivate, but do not replace, the planned GPU transport and end-to-end ablations. The manifest, 100 per-cell raw trials, structured check outcomes, generated table rows, and SHA-256 checksums are archived under `artifacts/trajectory_ir/cpu_fd344ad_macos_arm64/`.

8.4 RQ4: Exploratory bounded-staleness observations

The prespecified E6 study separated the device-allocation contrast—colocated synchronous execution (`S0`) versus separate-slab synchronous execution without overlap (`D0`)—from the onset of generation–training overlap—separate-slab synchronous (`D0`) versus separate-slab asynchronous execution with overlap (`A1`)—followed by weight-publication and staleness-allowance sweeps. The formal grid sweep was canceled on 2026-09-21 after preliminary runs; only single-seed capacity measurements and a separate quality-target pilot were

completed. The capacity timings, obtained with evaluation disabled, serve as planning observations rather than a full-budget, equal-quality comparison and are not used for a headline speedup claim.

The quality-target pilot did not pass. The `DO` pilot's reported MATH-500 avg@4 decreased from 0.798 at rollout 150 to 0.773 at rollout 300, contrary to the prespecified requirement that the later checkpoint improve. No quality threshold was established and no time-to-quality result is reported. Training reward also declined over the later window. This is a negative outcome for the tested pilot criterion; it neither identifies asynchrony as the cause nor establishes that every bounded-staleness configuration is ineffective. Two checkpoints are insufficient to locate a precise turning point.

Evaluation variability and requirements for a restart. Repeated evaluation of a single checkpoint exhibited noticeable prompt-level avg@4 variance despite a small aggregate difference. That observation is specific to the sampled checkpoint and decoding protocol; it is not a universal noise floor or a substitute for training-seed replication. A renewed study would first audit scorer behavior, request-level sampling, and repeat-evaluation variability; match the synchronous/asynchronous estimator and learner settings; and record complete driver time and buffer admission, rejection, discard, retry, and completion accounting. The trainer's internal `perf/step_time_s` metric alone omits outer-loop weight publication, evaluation, and checkpoint work. Only after establishing a suitable learning budget and a separately piloted target should a replicated formal grid support a lag–quality–time conclusion.

8.5 Fairness and statistical protocol

Effective work is matched before wall-clock time: prompts, samples per prompt, decoded resolution or maximum tokens, rollout steps, replayed steps/tokens, micro-batching semantics, and optimizer updates. Primary full-system metrics count all allocated training, rollout, and reward resources on both sides. Reward placement and concurrency must be disclosed; a reward-excluded number is supplementary and carries a separate service-cost report. We prespecify warm-up removal, use at least 20 steady-state iterations for throughput summaries, and report the full distribution rather than a single minimum. Training summaries use the seed as the statistical unit. The present figures show the available individual-seed summaries; a full release should additionally report justified seed-level uncertainty. Bootstrap intervals over prompts or images must be labeled separately and cannot replace seed uncertainty. The complete systems release must show all process repetitions and the post-warm-up per-rollout distribution; uncertainty of a speed ratio must use repetitions, not correlated rollouts, as its independent units. E3-AR releases its 499-rollout post-warm-up distribution, but has only one process-level run per framework; the older report-derived E3 summaries provide neither the complete per-rollout distribution nor independently revalidated repetitions. The final checkpoint and named primary endpoint are fixed in advance; the auxiliary metrics diagnose failure modes and are not alternative endpoints chosen after seeing results. Any future E6 time-to-quality study must use MATH-500 and a target prespecified from a separate pilot before the formal grid, bracket first observed crossings by evaluation times, and censor runs that never cross. The canceled study produced no such result. Elapsed time spans initialization, training, publication, checkpointing, and evaluation; steady-state throughput is reported separately. All published cells require complete scoring or an explicit missingness analysis; silently averaging only successful evaluator responses is inadmissible.

9 Related Work

LLM post-training systems. HybridFlow/veRL combines single- and multi-controller execution and introduces hierarchical APIs for mapping RLHF dataflows to devices (Sheng et al., 2024). OpenRLHF uses Ray, vLLM, and DeepSpeed to schedule the models in an RLHF workflow (Hu et al., 2024). ROLL emphasizes a single-controller abstraction, parallel workers, sample lifecycle scheduling, and flexible device mapping (Wang et al., 2025). NeMo-RL provides scalable training and generation backends and multimodal model support (NVIDIA Corporation, 2025). Group-relative recipes in these systems (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025) typically record one level of sampling structure—responses grouped under their prompt by a per-prompt identifier or by contiguous sample order—whereas UniRL generalizes that group key to multi-level ancestry encoded in sample identifiers. These systems

already drive distinct generation and training backends from one control loop and colocate or separate those backends as a placement choice; UniRL keeps that split and changes the object the loop schedules. Scheduling and DP sharding move complete root groups rather than rows, and each generated layer carries the replay segment for its own AR or diffusion stage (Section 3). A flat-row batch can recover the same group memberships by joining on retained parent keys (Section 2); UniRL differs in that sharding, credit assignment, and cross-stage conditioning consume that ancestry in place, so changing the execution path does not require rebuilding the join. RQ3 specifies the corresponding control: an ID-joined reference on recorded rewards and replay tensors (Section 8.3). To date its agreement check (E4a) uses synthetic fixtures, and the paired cost of carrying the structure (E5-R) is unmeasured; the single-process CPU regression is not that measurement.

Asynchronous post-training. AReaL fully decouples generation and training and controls staleness to improve language-reasoning throughput (Fu et al., 2025). Relax uses service-level separation, a TransferQueue data bus, and configurable staleness for omni-modal post-training (Zhang et al., 2026). slime supports persistent asynchronous rollout generation around a training/rollout/data-buffer loop (Zhu et al., 2025). These systems make asynchrony an execution design space. UniRL takes a conservative point within it: weight publication remains a barrier, and lag is bounded in optimizer-update units (Section 4.5). Each generated layer carries its behavior version in the trajectory IR, while admission, lag filtering, and weight publication stay in one manager shared by the AR and diffusion trainers. The filter keeps or drops whole root groups so that group-relative baselines stay complete, and no training batch spans behavior versions; this is stricter than AReaL, whose decoupled objective admits trajectories generated under several behavior versions (Fu et al., 2025). RQ4 reports only single-seed capacity measurements and a quality-target pilot that did not pass; it supports no throughput or time-to-quality comparison with these systems (Section 8.4).

Multimodal and diffusion post-training systems. VeRL-Omni extends veRL toward diffusion, unified, and omni-modality models and integrates vLLM-Omni rollout (Huang et al., 2026). Relax is explicitly omni-native (Zhang et al., 2026). Modality coverage therefore does not distinguish UniRL from these systems. The distinction is how ancestry and typed replay segments survive execution: an AR stage and a diffusion stage are two layers of one `Sample` (Section 3), and tensor references move their dense state without changing that lineage (Section 4). Models and algorithms meet only at the per-stage conditions, replay segment, and advantages that an algorithm consumes, so N_{mod} models and N_{alg} shared algorithms require $N_{\text{mod}} + N_{\text{alg}}$ rather than $N_{\text{mod}} \times N_{\text{alg}}$ integrations (Section 3.6). E4 runs one frozen AR rewrite stage into a trained diffusion stage (Section 8.3). Its lineage check (E4a) uses synthetic fixtures, its quality comparison (E4b) is inconclusive across three seeds, and its rewrite text was malformed; the module inventory documents what that path reuses rather than measuring human integration effort.

Diffusion RL algorithms. DDPO casts denoising as a multi-step decision process (Black et al., 2023). Flow-GRPO converts flow sampling to an SDE and applies group-relative optimization (Liu et al., 2025); DanceGRPO generalizes related ideas across visual generation tasks (Xue et al., 2025); DiffusionNFT instead optimizes a forward-process objective from reward-ranked generations (Zheng et al., 2025). UniRL consumes Flow-GRPO-style and DiffusionNFT objectives through its algorithm interface and claims neither objective (Section 5). What the interface contributes is that their incompatible sufficient statistics stay explicit: Flow-GRPO-style objectives replay recorded SDE transitions, DiffusionNFT trains on clean generated samples, and both reuse the same train stack for micro-batching and optimizer state.

10 Limitations and Open Evidence

Integration cost is per architecture. Adding an architecture requires one bundle with its stages, conditions, and rollout adapters; this cost is paid once per model rather than once per model–algorithm pair. Architecture-specific details such as tokenizers, noise schedulers, latent layouts, and LoRA targets stay inside that bundle and its adapters rather than being abstracted by the trajectory type. The ordered-layer representation targets trajectories that grow by forking and observation; arbitrary ragged forests are outside its current scope.

Topology is configured, not optimized. Placement scopes make resource choices explicit but do not search for the best split. The best topology depends on model memory, reward cost, rollout variance, interconnect, and publication size.

Asynchronous execution is bounded and partially specialized. Weight publication is a barrier, and current AR and diffusion paths impose different admissible in-flight structure. Fault isolation is limited by the shared Ray runtime and role lifecycle; the system does not yet claim service-level recovery comparable to dedicated service architectures.

Case-study scope. The reward-serving and rollout-fidelity case studies (Sections 6 and 7) evaluate targeted mechanisms outside the E0–E6 program on a single BAGEL image-editing workload with small rollout counts per arm. In Section 6.4, the image-editing wall-clock durations are rescaled estimates that combine measured reward-side traces with the generation speedup from restored source conditioning; in Section 6.5 and Section 7, the MPS colocation runtime, token-layout check, per-group canvas, sibling packing, and cost-aware load balancer are evaluated from open UniRL pull requests (#492, #547–#550) or an experimental branch rather than the v0.2.0 release commit.

The empirical case is incomplete. The GPU evidence supports qualified observations about diffusion scores and diversity, native-recipe iteration time, deployment sensitivity, and one recipe-matched AR throughput comparison. The AR comparison releases per-rollout evidence but lacks independent process repetitions and a common external quality endpoint; it therefore does not establish run-to-run uncertainty, training-quality reproduction, or time-to-quality. The combined evidence also does not establish a quality advantage from lineage grouping, bounded IR overhead, scaling, or a time-to-quality benefit from asynchrony. Raw GPU records remain unavailable for the legacy report-derived studies, which still need exact run manifests, consistent aggregate provenance, and independent regeneration. The malformed rewrite pipeline and synthetic-only E4a fixtures further limit the evidence for the central cross-stage claim.

11 Conclusion

UniRL treats post-training as a trajectory program whose semantic object survives changes in generative family and execution topology. Lineage, typed modality values, replay segments, reward, credit, and behavior version live in one IR; rollout, reward, training, transport, and weight publication are logical roles mapped separately to resources. Two case studies on instruction-guided image editing illustrate how these boundaries operate in practice: micro-batched generation–reward overlap hides remote judge latency when the judge has spare capacity, while structural token-layout checks and sibling request packing across the rollout-engine boundary recover conditioning fidelity and reduce generation time. In the main evaluation program, the current evidence shows diffusion score improvements accompanied by reduced diversity, faster iteration of one native diffusion configuration, a substantial penalty for the measured separate deployment, and 8.2% higher end-to-end output-token throughput for UniRL in one recipe-matched Qwen3-4B-Base GRPO run. That AR result is a workload-specific throughput observation rather than a repeated-run or time-to-quality conclusion. The full trajectory-centric claim remains open until real-rollout lineage validation and matched representation-cost measurements complement those observations, and bounded staleness remains an exploratory operating-region question with no demonstrated time-to-quality benefit in the canceled study.

Reproducibility statement

The core architectural characterization in Sections 3–5 refers to UniRL release v0.2.0, commit `fd344ad6036a0ba7be9900a9d0cf78ebf59fa738` (2026-10-08). The companion report repository records the claim-to-evidence plan (`PAPER_PLAN.md`), the cluster execution protocol and configuration-parity gates (`EXPERIMENT_RUNBOOK.md`), the operational handoff commands (`GPU_HANDOFF_README.md`), and the result audit (`EXPERIMENT_RESULTS.md`). The legacy report bundle `artifacts/reported_results/2026-09-22/` (integrating the 2026-09-22 experiment summary and companion report pull request #2 at commit `ecce5097`) stores compact source snapshots, source-level caveats, reported numerical inputs, and gen-

erated tables and figures. These reproduce the transcription and presentation of the legacy reports rather than the original raw GPU logs. The reward-serving and rollout-fidelity case studies (Sections 6 and 7) archive their per-rollout records, rescaling models, and table/figure generators under `artifacts/reported_results/2026-09-30-reward-service/` (UniRL commit `651490e` and UniRL pull request #492 at `8bb4119`) and `artifacts/reported_results/2026-10-04-rollout-fidelity/` (UniRL pull requests #547–#550). The separately rerun AR systems bundle in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/` pins UniRL commit `6148a5d`, the veRL reference and compatibility commits, both resolved recipes, the recipe and effective-work audit, run identities, 499 post-warm-up records per framework, phase and band summaries, and the plotting script. Commit `6148a5d` is on a pre-release branch: it descends from an earlier audit commit and carries four rollout-performance and decoding commits not included in v0.2.0, so E3-AR measures that branch rather than the audited v0.2.0 tag. Its UniRL run is W&B run `2x4tdjwa`; its veRL run is `grpo-verl-natural500`, whose per-rollout records come from the trainer's console metrics. The HunyuanImage-3.0 and MiniMax-H3 bundles in `artifacts/reported_results/2026-09-30-hunyuanimage3/` and `artifacts/reported_results/2026-09-30-minimax-h3/` store the resolved recipes, W&B run identities, direct per-rollout history exports, and rendered learning curves. All graphics are committed PDF assets included by relative paths, so Overleaf does not require Python or shell escape to build the manuscript. The preliminary RQ3 CPU evidence is regenerated by `experiments/trajectory_ir/run_cpu_evidence.py`; its manifest pins the source state, command, environment, workload, warm-up, repetitions, and timer.

References

Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. 2023. URL https://arxiv.org/abs/2305.13301.

Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. Emerging properties in unified multimodal pretraining. 2025. URL https://arxiv.org/abs/2505.14683.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. 2025. URL https://arxiv.org/abs/2505.24298.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. 2024. URL https://arxiv.org/abs/2405.11143.

Yongxiang Huang, Cheung Kawai, Jingan Zhou, Yingshu Chen, openYuanrong Team, and Xibin Wu. VeRL-Omni: Easy, fast, and stable RL training for diffusion and omni-modality models. GitHub repository, 2026. URL https://github.com/verl-project/verl-omni.

Amita Kamath, Kai-Wei Chang, Ranjay Krishna, Luke Zettlemoyer, Yushi Hu, and Marjan Ghazvininejad. GenEval 2: Addressing benchmark drift in text-to-image evaluation. 2025. URL https://arxiv.org/abs/2512.16853.

Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2305.01569.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. 2025. URL https://arxiv.org/abs/2505.05470.

Xin Luo, Jiahao Wang, Chenyuan Wu, Shitao Xiao, Xiyan Jiang, Defu Lian, Jiajun Zhang, Dong Liu, and Zheng Liu. EditScore: Unlocking online RL for image editing via high-fidelity reward modeling. 2025. URL https://arxiv.org/abs/2509.23909.

Qianli Ma, Yaowei Zheng, Zhelun Shi, Zhongkai Zhao, Bin Jia, Ziyue Huang, Zhiqi Lin, Youjie Li, Jiacheng Yang, Yanghua Peng, Zhi Zhang, and Xin Liu. VeOmni: Scaling any modality model training with model-centric distributed recipe zoo. 2025a. URL https://arxiv.org/abs/2508.02317.

Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards wide-spectrum human preference score. 2025b. URL https://arxiv.org/abs/2508.03789.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

NVIDIA Corporation. NVIDIA multi-process service. Documentation. URL https://docs.nvidia.com/deploy/mps/latest/.

NVIDIA Corporation. NeMo RL: A scalable and efficient post-training library. GitHub repository, 2025. URL https://github.com/NVIDIA-NeMo/RL.

Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. 2024. URL https://arxiv.org/abs/2407.00079.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021. URL https://arxiv.org/abs/2103.00020.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL https://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. 2024. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. 2024. URL https://arxiv.org/abs/2409.19256.

Stability AI. Stable diffusion 3.5 medium model card. Hugging Face model repository, 2024. URL https://huggingface.co/stabilityai/stable-diffusion-3.5-medium.

Weixun Wang et al. ROLL: An efficient and user-friendly scaling library for reinforcement learning with large language models. 2025. URL https://arxiv.org/abs/2506.06122.

Keming Wu, Sicong Jiang, Max Ku, Ping Nie, Minghao Liu, and Wenhu Chen. EditReward: A human-aligned reward model for instruction-guided image editing. 2025. URL https://arxiv.org/abs/2509.26346.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. 2023. URL https://arxiv.org/abs/2306.09341.

Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2304.05977.

Zeyue Xue, Jie Wu, Yu Gao, Fangyuan Kong, Lingting Zhu, Mengzhao Chen, Zhiheng Liu, Wei Liu, Qiushan Guo, Weilin Huang, and Ping Luo. DanceGRPO: Unleashing GRPO on visual generation. 2025. URL https://arxiv.org/abs/2505.07818.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. 2025. URL https://arxiv.org/abs/2505.09388.

Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, and Hongsheng Liu. vLLM-Omni: Fully disaggregated serving for any-to-any multimodal models. 2026. URL https://arxiv.org/abs/2602.02204.

Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. 2025. URL https://arxiv.org/abs/2503.14476.

Liujie Zhang, Benzhe Ning, Rui Yang, Xiaoyan Yu, Jiaxing Li, Lumeng Wu, Jia Liu, Minghao Li, Weihang Chen, Weiqi Hu, and Lei Zhang. Relax: An asynchronous reinforcement learning engine for omni-modal post-training at scale. 2026. URL https://arxiv.org/abs/2604.11554.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://arxiv.org/abs/1801.03924.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Bali, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. 2023. URL https://arxiv.org/abs/2304.11277.

Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. 2025. URL https://arxiv.org/abs/2509.16117.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, and Joseph E. Gonzalez. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2312.07104.

Zilin Zhu, Chengxing Xie, Xin Lv, and contributors. slime: An LLM post-training framework for RL scaling. GitHub repository, 2025. URL https://github.com/THUDM/slime.

A Claim-to-source audit

Table 14 records the executable source used to support mechanism descriptions. It does not prove performance or successful execution for every configuration.

Table 14: Primary code evidence at commit `fd344ad` (v0.2.0).

Mechanism	Primary source symbols
Trajectory IR	`unirl/types/sample.py`: `Part`, `Sample`; `unirl/types/primitives.py`; `unirl/types/segments/`
Batch algebra	`unirl/distributed/tensor/batch.py`: `FieldKind`, `Batch`
Tensor references	`unirl/distributed/tensor/ref.py`: `TensorSpan`, `TensorRef`; `unirl/distributed/tensor/transport.py`: `TensorTransport`
Roles and placement	`unirl/distributed/group/remote.py`: `Remote`; `device_pool.py`, `placement.py`, `handle.py`, `dispatch.py`
Rollout boundary	`unirl/rollout/engine/base.py`: `BaseRolloutEngine`; concrete engines under `unirl/rollout/engine/`, including `vllm/`; `unirl/reward/stack.py` for overlapped scoring
Training boundary	`unirl/algorithms/base.py`: `StageAlgorithm`; `unirl/train/stack/base.py`: `TrainStack`; `anchor.py`, `planner/update.py`
Sync trainers	`unirl/trainer/ar.py`: `ARTrainer`; `unirl/trainer/diffusion.py`: `DiffusionTrainer`; `pe.py`; `residency.py`
Async and agentic	`unirl/trainer/async_rollout.py`, `async_ar.py`, `async_diffusion.py`, `agentic.py`; `unirl/rollout/manager/`
Weight publication	`unirl/distributed/weight_sync/`, including `full/ckpt_engine_ipc.py`; engine-specific receivers under `unirl/rollout/engine/`
Instrumentation	`unirl/utils/profiling.py`, `memory_monitor.py`, `wandb_logger.py`

B Colocated residency orders

Table 15 lists the orders that the trainers at the audited release use on a colocated slab. Each satisfies the two invariants of Section 4.3: a due publication completes before generation, and memory is released before the role that needs it is brought up.

Table 15: Colocated residency orders. "Offload" moves training state to host memory; "sleep" releases the engine's weight and cache memory, and "wake" restores it.

Order	Applies when	Before generation	After generation
Default	AR, agentic, and prompt-enhancement trainers, unless a row below applies	Wake the engine, publish weights if due, offload training state if enabled	Sleep the engine, then restore training state
Split wake	AR; publication due, offload enabled, engine can restore weight memory separately	Wake weight memory only, publish weights, offload training state, complete the wake	As default
Split wake, vLLM weight publication	As above, publishing through vLLM's native weight-transfer engine	Offload optimizer state, wake weight memory, publish weights, offload the model shard, complete the wake	As default
Offload without publication	AR; offload enabled, no publication due	Offload training state, then wake the engine	As default
Extract, then publish	AR with the engine pinned to one designated GPU of the slab; unified-model trainer with separate AR and diffusion engines	Extract weights if due, offload training state if enabled, wake the engine(s), publish the extracted weights	As default; a pinned AR engine with offload disabled stays awake
Per-role residency	Diffusion trainers	Extract the adapter first if training will be offloaded; offload every non-resident role, wake the engine, publish weights if due	Sleep the engine unless the rollout role is resident; restore training state before the optimizer step

C Required run artifact schema

Every result row must resolve to a run directory containing:

- source and baseline commit hashes, submodule hashes, and dirty patches;
- the launch command and fully resolved configuration;
- model, reward, dataset, and tokenizer revisions or content hashes;
- GPU model/count, node/interconnect description, driver, CUDA, PyTorch, generation-engine, and training-backend versions;
- raw stdout/stderr, per-rollout structured timing, memory and utilization traces, checkpoint metadata, and evaluation outputs;
- seed, warm-up exclusion rule, failure/retry accounting, and statistical aggregation script.

For training runs, serialized trace samples must redact private data while retaining lineage IDs, shapes, dtypes, behavior versions, reward components, and segment index metadata. For system baselines, an alignment manifest must explain every nonidentical setting and whether it changes effective work.

D Correctness status and remaining checks

Executed locally. The preliminary RQ3 harness performs ten fail-fast checks on the audited source: lineage-valid cross-stage construction; exact split/concat; tree-complete select and slice; mean reward propagation plus group-normalized advantages; rejection of bad lineage, modality keys, and primitive batch sizes; packed variable-length text select/concat; and lazy tensor-reference selection, materialization, identity, and bounds behavior. The structured outcomes and data-sensitive SHA-256 round-trip hash are stored with the run artifacts.

Required before complete evidence admission. The subsequent reports describe E0 correctness closure, but its underlying test and configuration artifacts were not revalidated in this integration. The following system-level checks are the evidence requirements, not a claim that every experiment still lacks each check:

1. **Rollout–replay parity:** before the first update, compare engine-emitted and train-side replay log probabilities or transition statistics under stated numerical tolerances.
2. **Publication parity:** repeat the parity check after each LoRA or full-weight publication path used in an experiment.
3. **Batch-version integrity:** verify that an optimizer batch never mixes behavior versions when the configured algorithm assumes one version; this is meaningful only on the asynchronous and agentic paths, whose behavior versions advance at publication.
4. **Boundary integrity:** checkpoint and evaluation begin only after the async manager is empty; resume restores the optimizer version and republishes it before new rollout.
5. **Work accounting:** compare generated tokens or pixels, denoising and replay steps, scored samples, and committed optimizer updates across systems.
6. **Structural preconditions:** validate unique root IDs, equal layer counts, parent-major grouping, shared-field equality, and behavior-version provenance before merging or propagating credit; check against an ID-joined reference.